

OFDM PHY Layer Implementation based on the 802.11a Standard and System Performance Analysis

Bachelor thesis performed in Electronics Systems
by

Luis Zarzo Fuertes

LITH-ISY-EX-ET--05/0307--SE
2005-03-22

**OFDM PHY Layer Implementation based on the 802.11a Standard
and System Performance Analysis**

Bachelor Thesis in Electronic Systems at Linköping University
Institute of Technology
by

Luis Zarzo Fuertes

LiTH-ISY-EX-ET--05/0307--SE
2005-03-22

Supervisor: Kent Palmkvist
Examiner: Kent Palmkvist

	Avdelning, Institution Division, Department Institutionen för systemteknik 581 83 LINKÖPING	Datum Date 2005-03-03
---	---	------------------------------------

Språk Language Svenska/Swedish X Engelska/English	Rapporttyp Report category Licentiatavhandling X Examensarbete C-uppsats D-uppsats Övrig rapport _____	ISBN	
		ISRN LiTH-isy-ex-et--05/0307--SE	
		Serietitel och serienummer Title of series, numbering	ISSN _____

URL för elektronisk version
<http://www.ep.liu.se/exjobb/isy/2005/307/>

Titel Title	OFDM PHY Layer Implementation based on the 802.11 a Standard and system performance analysis
Författare Author	Luis Zarzo Fuertes

Sammanfattning
Abstract

Wireless communication is facing one of the fastest developments of the last years in the fields of technology and computer science in the world. There are several standards that deal with it. In this work, the IEEE standard 802.11a, which deals with wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, is going to be discussed in detail.

Taking this into consideration, PHY specifications and its environment are going to be studied.

The work that the ISY department at the Institute of Technology of the Linköping University has proposed is to design a PHY layer implementation for WLANs, in a CPU, using MATLAB/Simulink and in a DSP processor, using Embedded Target for C6000 DSP and Code Composer Studio and, once implemented both, to perform and analyse the performance of the system under those implementations.

Nyckelord
Keyword

Wireless communication, Standard 802.11a, physical layer, medium access control

ABSTRACT

Wireless communication is facing one of the fastest developments of the last years in the fields of technology and computer science in the world. There are several standards that deal with it. In this work, the IEEE standard 802.11a, which deals with wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, is going to be discussed in detail.

Taking this into consideration, PHY specifications and its environment are going to be studied.

The work that the ISY department at the Institute of Technology of the Linköping University has proposed is to design a PHY layer implementation for WLANs, in a CPU, using MATLAB/Simulink and in a DSP processor, using Real-Time Workshop and Code Composer Studio and, once implemented both, to perform and analyse the performance of the system under those implementations.

ACKNOWLEDGEMENTS

English

This thesis work is the end of four years of work at Alcalá de Henares University and one year and a half at the Linköping University Institute of Technology.

First of all I would also like to thank the University of Linköping and my home University, University of Alcalá the Henares, for giving me the opportunity of studying and writing this final thesis. I would like to recognize the job my coordinator in Spain, Antonio Guerrero, do. I wish to thank my supervisor Kent Palmkvist for giving me the opportunity of doing this thesis in the Department of Electrical Engineering (ISY) and for his help and advise during the project.

I will never forget my class mates for all those great and bad moments we spent together throughout the degree at the University. And all my friends in general for always being there for everything I need.

Thanks most of all to my parents and specially my brother, Oscar, for their infinite support all this years and Julia for being always with me during the last four months.

Finally I would like to say that I will never forget the city of Linköping and Sweden, it has been a wonderful experience to be there during one year and a half and to spend this time with a lot of extraordinary people that I have known during my stay in Sweden.

Luis Zarzo Fuertes

Linköping, Sweden

March, 2005

Spanish

Este proyecto fin de carrera representa el final de cuatro años de estudio en la universidad de Alcalá de Henares y un año y medio en la Universidad de Linköping.

Primeramente me gustaría dar las gracias a las dos universidades por ofrecerme la oportunidad de estudiar y de realizar mi proyecto final en el extranjero. Me gustaría, a su vez, reconocer el trabajo que realiza mi coordinador en España, Antonio Guerrero. Desearía dar las gracias a mi supervisor en Suecia, Kent Palmkvist por darme la oportunidad de realizar este proyecto en el departamento de Ingeniería Electrónica (ISY) y por su ayuda y sus recomendaciones durante el desarrollo del proyecto.

Nunca olvidaré a mis compañeros de la universidad por todos los buenos y malos momentos que hemos pasado juntos. Y a todos mis amigos en general por estar siempre a mi lado cuando los he necesitado.

Agradecer sobre todo a mis padres y especialmente a mi hermano, Oscar, por su enorme apoyo durante todos estos años y a Julia por estar siempre a mi lado durante estos últimos cuatro meses.

Finalmente me gustaría decir que nunca olvidaré ni la ciudad de Linköping ni Suecia, Ha sido una experiencia maravillosa estar allí durante este último año y medio y haber pasado este tiempo con un montón de personas extraordinarias, las cuales he tenido la suerte durante mi estancia en Suecia.

Luis Zarzo Fuertes

Linköping, Sweden

Marzo, 2005

CONTENTS

1. INTRODUCTION	1
1.1 MAIN GOAL	1
1.2 PLANNING	1
1.3 RESOURCES	2
2. INTRODUCTION TO WIRELESS NETWORKS	3
2.1 WIRELESS TECHNOLOGY	3
2.2 WIRELESS NETWORKS	4
2.3 WIRELESS LANs	4
2.4 WIRELESS STANDARDS	5
2.5 OSI	7
2.6 INTRODUCTION TO 802.11 a STANDARD	8
3. SIMULINK IMPLEMENTATION	11
3.1 ABOUT THE SIMULATION	11
3.2 STRUCTURE OF THE SYSTEM	12
3.3 OFDM PHY LAYER MODEL USED	13
3.4 MODEL DESCRIPTION	14
3.5 SIMULATION RESULTS AND ANALYSIS	33
4. DIGITAL SIGNAL PROCESSING	37
4.1 INTRODUCTION TO DSP	37
4.2 HOW DSPs ARE DIFFERENT FROM OTHER MICROPROCESSORS	38
4.3 INTRODUCTION TO THE TMS320C6000 PLATFORM OF DIGITAL SIGNAL PROCESSORS	39
4.4 DSP IMPLEMENTATION	42
5. FFT IMPLEMENTATION & PERFORMANCE ANALYSIS	65
5.1 FFT IMPLEMENTATION	65
5.2 ANALYSIS	69
6. FINAL CONCLUSION AND FUTURE WORK	85
7. ABBREVIATIONS AND ACRONYMS	87
8. REFERENCES	89
9. APPENDIX	91
9.1 APPENDIX 1	91
9.2 APPENDIX 2	93

INDEX OF FIGURES

Figure 1. The infrastructure network	6
Figure 2. Ad Hoc Topology	6
Figure 3. Network architecture based on the OSI model	7
Figure 4. PLCP Transmit Procedure	9
Figure 5. Simulation parameters	12
Figure 6. OFDM PHY Model	14
Figure 7. Modulator subsystem	17
Figure 8. Convolutional encoder(K=7)	18
Figure 9. 64-QAM constellation points	20
Figure 10. Constellation diagram after modulation	21
Figure 11. 48 subcarriers, 4 of them pilots	22
Figure 12. OFDM Transmitter subsystem	23
Figure 13. Subcarrier frequency allocation	24
Figure 14. FFT Mapping	25
Figure 15. IFFT/FFT description	26
Figure 16. Orthogonal multicarrier technique	27
Figure 17. Cyclic prefix	28
Figure 18. OFDM receiver	30
Figure 19. Demodulator	30
Figure 20. Received signal SNR = 50	33
Figure 21. Received signal SNR = 20	34
Figure 22. Received signal SNR = 10	34
Figure 23. BER/SNR Graphic	35
Figure 24. Typical components of a DSP system	37
Figure 25. TMS320C6000 block diagram	40
Figure 26. Architecture of the TMS320C6713 DSP	42
Figure 27. Development cycle	43
Figure 28. CCS Components	43
Figure 29. Project structure	44
Figure 30. System Target File Browser	48
Figure 31. Target error	48
Figure 32. CCS Setup	49
Figure 33. Simulation Parameters	52
Figure 34. Parameters for the subsystem to build	53
Figure 35. RTW builder error	54
Figure 36. DSP/BIOS File	58
Figure 37. Signal generated by the equation	66
Figure 38. Overview of the project	68
Figure 39. Matlab profiler	69

Figure 40. Profile report	70
Figure 41. Dspblkfft2 function details	71
Figure 42. Profiler properties	73
Figure 43. Breakpoints	74
Figure 44. Profiler results	76
Figure 45. Clock Setup	77
Figure 46. Matlab Benchmark	79
Figure 47. Before running the Simulink model	80
Figure 48. While running the Simulink model	81
Figure 49. Before running the FFT algorithm	81
Figure 50. While running the FFT algorithm	82
Figure 51. CPU usage of simulink model	82
Figure 52. CPU load Graph	83

INDEX OF TABLES

Table 1. Rate-dependent parameters of 802.11a	15
Table 2. Global parameters	16
Table 3. Profiler results	75

1. INTRODUCTION

1.1 MAIN GOAL

The main goal of this thesis work is to learn and understand the features of the IEEE standard 802.11a and afterwards, once familiarised with this standard, to develop an OFDM 802.11a PHY layer baseband implementation with the characteristics showed in such standard. Matlab/Simulink is used for designing the model and Real-Time Workshop and Code Composer Studio for targeting that Simulink model on the DSP board in order to have a better understanding of the standard and of the system performance running this model under both platforms.

1.2 PLANNING

The following steps have been followed for developing this thesis work:

- First of all, the IEEE standard 802.11a was read and understood in order to obtain enough knowledge about the features of the standard mainly and also to focus on the most important topics; these are OFDM modulation and FFT.
- Understanding what DSP is and what is the use of a DSP board.
- Once the necessary knowledge about what should be implemented have been obtained, the next step is to familiarise ourselves with the necessary tools for the implementation as well as developing some ideas about how the implementation of the model could be done, starting with Matlab, Simulink and Code Composer Studio.
- Having the proper knowledge, the Simulink implementation can be started. Such implementation consists of achieving the design of an OFDM PHY layer baseband and, once done, test it and obtain some results.

- After the finalisation and testing of the Simulink model, the next step was to read some manuals about how to use Real-Time Workshop, specially Embedded Target for c6000. With the know-how for using such tool, the Simulink model is targeted into the DSP board.
- The final step is, once implemented the model in both applications, is to achieve an analysis of the system performance.

1.3 RESOURCES

The resources used to develop this thesis work have been the following.

1.3.1 Software

- MATLAB v 6.5.1 (R13)
- Simulink v 5.1
- Real-Time Workshop.
- Embedded Target for C6000 DSP (Trial Version-2003)
- Code Composer Studio 'C6713 - v2.21
- Operating System: Windows XP home Edition

1.3.2 Hardware

- C6713 DSP Development Board 16MB SDRAM
- CPU, Intel Pentium III 450MHz, 128 MB RAM Memory

2. INTRODUCTION TO WIRELESS NETWORKS

The purpose of this section is to provide readers with a basic understanding of the wireless networks and specially Wireless LANs and of the 802.11a techniques, concepts, architecture and principles of operations, mainly based on the PHY layer[10].

2.1 WIRELESS TECHNOLOGY

Wireless technologies have become increasingly popular in our everyday business and personal lives. Personal digital assistants (PDA) allow individuals to access calendars, e-mail, address and phone number lists, and the Internet. Some technologies even offer global positioning system (GPS) capabilities that can pinpoint the location of the device anywhere in the world. Wireless technologies promise to offer even more features and functions in the next few years. An increasing number of government agencies, businesses, and home users are using, or considering using, wireless technologies in their environments.

Wireless technologies, in the simplest sense, enable one or more devices to communicate without physical connections, without requiring network or peripheral cabling. Wireless technologies use radio frequency transmissions as the means for transmitting and receiving data over the air, whereas wired technologies use cables. Wireless technologies range from complex systems, such as Wireless Local Area Networks (WLAN) and cell phones to simple devices such as wireless headphones, microphones, and other devices that do not process or store information. They also include infrared (IR) devices such as remote controls, some cordless computer keyboards and mice, and wireless hi-fi stereo headsets, all of which require a direct line of sight between the transmitter and the receiver to close the link.

Wireless communications offer organizations and users many benefits such as portability and flexibility, increased productivity, and lower installation costs. Wireless technologies cover a broad range of differing capabilities oriented toward different uses and needs. Wireless local area network (WLAN) devices, for instance, allow users to move their laptops from place to place within their offices without the need for wires and without losing network connectivity. Less wiring means greater flexibility, increased efficiency, and reduced wiring costs.

However, risks are inherent in any wireless technology. Some of these risks are similar to those of wired networks; some are exacerbated by wireless connectivity; some are new. Perhaps the most significant source of risks in wireless networks is that the technology's underlying communications medium, the airwave, is open to intruders, making it the logical equivalent of an Ethernet port in the parking lot.

The loss of confidentiality and integrity and the threat of denial of service (DoS) attacks are risks typically associated with wireless communications. Unauthorized users may gain access to agency systems and information, corrupt the agency's data, consume network bandwidth, degrade network performance, launch attacks that prevent authorized users from accessing the network, or use agency resources to launch attacks on other networks.

2.2 WIRELESS NETWORKS

Wireless networks serve as the transport mechanism between devices and among devices and the traditional wired networks (enterprise networks and the Internet). Wireless networks are many and diverse but are frequently categorized into three groups based on their coverage range: Wireless Wide Area Networks (WWAN), WLANs, and Wireless Personal Area Networks (WPAN). WWAN includes wide coverage area technologies such as 2G cellular, Cellular Digital Packet Data (CDPD), Global System for Mobile Communications (GSM), and Mobitex. WLAN, representing wireless local area networks, includes 802.11, HiperLAN, and several others. WPAN, represents wireless personal area network technologies such as Bluetooth and IR.

2.3 WIRELESS LANs

WLANs allow greater flexibility and portability than do traditional wired local area networks (LAN). Unlike a traditional LAN, which requires a wire to connect a user's computer to the network, a WLAN connects computers and other components to the network using an access point device. An access point communicates with devices equipped with wireless network adaptors; it connects to a wired Ethernet LAN via an RJ-45 port. Access point devices typically have coverage areas of up to approximately 100 meters. This coverage

area is called a cell or range. Users move freely within the cell with their laptop or other network device. Access point cells can be linked together to allow users to even “roam” within a building or between buildings.

2.4 WIRELESS STANDARDS

WLANs are based on the IEEE 802.11 standard, which the IEEE first developed in 1997. The IEEE designed 802.11 to support medium-range, higher data rate applications, such as Ethernet networks, and to address mobile and portable stations.

802.11 is the original WLAN standard, designed for 1 Mbps to 2 Mbps wireless transmissions. It was followed in 1999 by 802.11a, which established a high-speed WLAN standard for the 5 GHz band and supported 54 Mbps. Also completed in 1999 was the 802.11b standard, which operates in the 2.4 - 2.48 GHz band and supports 11 Mbps. The 802.11b standard is currently the dominant standard for WLANs, providing sufficient speeds for most of today’s applications. Because the 802.11b standard has been so widely adopted, the security weaknesses in the standard have been exposed. Another standard, 802.11g operates in the 2.4 GHz waveband, where current WLAN products based on the 802.11b standard operate.

2.4.1 802.11 Architecture

The standard defines two basic network topologies: the infrastructure network and the ad hoc network. The infrastructure network is meant to extend the range of the wired LAN to wireless cells. A laptop or other mobile device may move from cell to cell (from AP to AP) while maintaining access to the resources of the LAN. A cell is the area covered by an AP and is called a “basic service set” (BSS). The collection of all cells of an infrastructure network is called an extended service set (ESS). This first topology is useful for providing wireless coverage of building or campus areas. By deploying multiple APs with overlapping coverage areas, organizations can achieve broad network coverage. WLAN technology can be used to replace wired LANs totally and to extend LAN infrastructure.

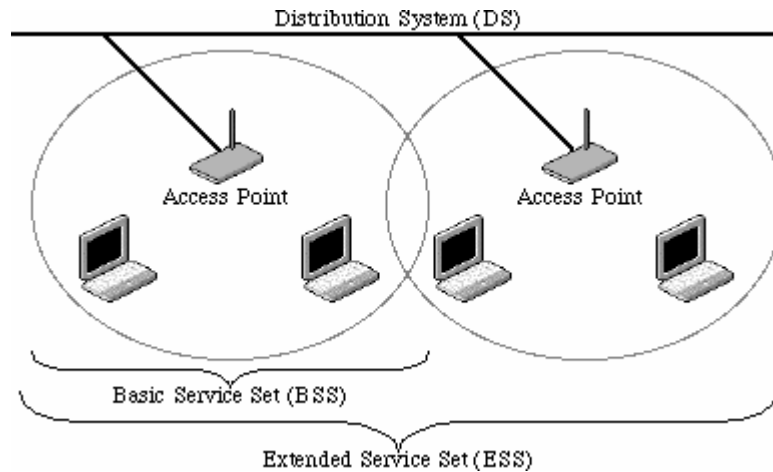


Figure 1: The infrastructure network.

A WLAN environment has wireless client stations that use radio modems to communicate to an AP. The client stations are generally equipped with a wireless network interface card (NIC) that consists of the radio transceiver and the logic to interact with the client machine and software. An AP comprises essentially a radio transceiver on one side and a bridge to the wired backbone on the other. The AP, a stationary device that is part of the wired infrastructure, is analogous to a cell-site (base station) in cellular communications. All communications between the client stations and between clients and the wired network go through the AP.

Although most WLANs operate in the “infrastructure” mode and architecture described above, another topology is also possible. This second topology, the ad hoc network, is meant to easily interconnect mobile devices that are in the same area (e.g., in the same room). In this architecture, client stations are grouped into a single geographic area and can be Internet-worked without access to the wired LAN (infrastructure network). The interconnected devices in the ad hoc mode are referred to as an Independent Basic Service Set (IBSS).

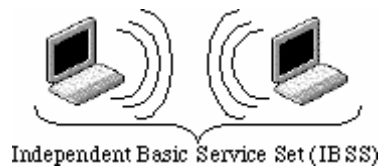


Figure 2: Ad Hoc Topology

The ad hoc configuration is similar to a peer-to-peer office network in which no node is required to function as a server. As an ad hoc WLAN, laptops, desktops and other 802.11 devices can share files without the use of an AP.

2.5 OSI

The International Standards Organization (ISO) defined an architecture called Open Systems Interconnection (OSI). The OSI reference model consists of seven protocol layers which are defined as follow[9]:

- 1) Application: Provides different services to the applications
- 2) Presentation: Converts the information
- 3) Session: Handles problems that are not communications issues
- 4) Transport: Provides end-to-end communication control
- 5) Network: Routes the information in the network
- 6) Data Link: Provides error control between adjacent models.
- 7) Physical: Connects the entity to the transmission media

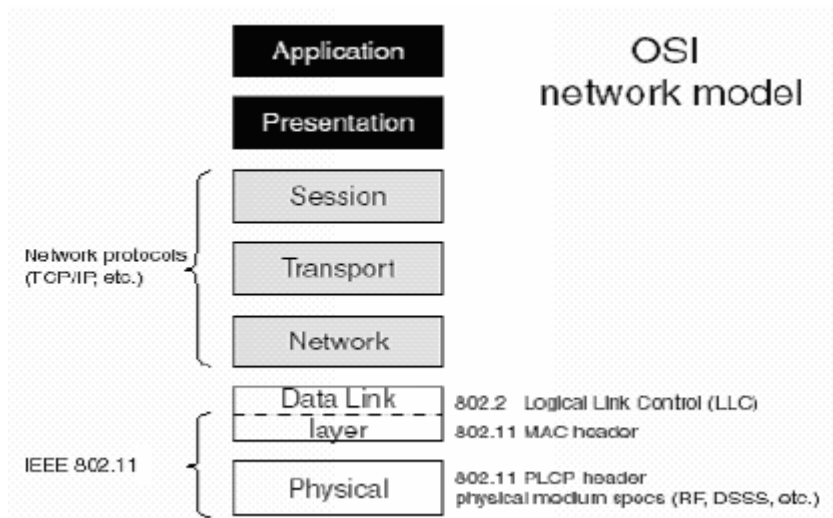


Figure 3. Network architecture based on the OSI model

The IEEE 802.11 standard focuses on the bottom two layers of the OSI model: data link layer and physical layer.

2.6 INTRODUCTION TO 802.11 a STANDARD

IEEE 802.11 standard specifies a 2.4 GHz operating frequency with data rates of 1 and 2 Mbps using either Direct Sequence Spread Spectrum (DSSS) or Frequency Hopping Spread Spectrum (FHSS). The IEEE 802.11a standard [23] specifies an OFDM physical layer (PHY) that splits an information signal across 52 separate subcarriers to provide transmission of data at a rate of 6, 9, 12, 18, 24, 36, 48, or 54 Mbps. In the 802.11a IEEE standard the 6, 12, and 24 Mbps data rates are mandatory. Four of the subcarriers are pilot subcarriers that the system uses as a reference to disregard frequency or phase shifts of the signal during transmission.

In the 802.11a standard, a pseudo binary sequence is sent through the pilot subchannels to prevent the generation of spectral lines. In the 802.11a, the remaining 48 subcarriers provide separate wireless pathways for sending the information in a parallel fashion. The resulting subcarrier frequency spacing in the IEEE 802.11a standard is 0.3125 MHz (for a 20 MHz bandwidth with 64 possible subcarrier frequency slots).

Also in the 802.11a standard, the primary purpose of the OFDM PHY is to transmit Media Access Control (MAC) Protocol Data Units (MPDUs) as directed by the 802.11 MAC layer. The OFDM PHY of the 802.11a standard is divided into two elements: the Physical Layer Convergence Protocol (PLCP) and the Physical Medium Dependent (PMD) sublayers.

The MAC layer of 802.11a standard communicates with the PLCP via specific primitives through a PHY service access point. When the MAC layer instructs, the PLCP prepares MPDUs for transmission. The PLCP also delivers incoming frames from the wireless medium to the MAC layer. The PLCP sublayer minimizes the dependence of the MAC layer on the PMD sublayer by mapping MPDUs into a frame format suitable for transmission by the PMD.

Under the direction of the PLCP, the PMD provides actual transmission and reception of PHY entities between two stations through the wireless medium. To provide this service, the PMD interfaces directly with the air medium and provides modulation and demodulation of the frame transmissions. The PLCP and PMD communicate using service primitives to govern the transmission and reception functions.

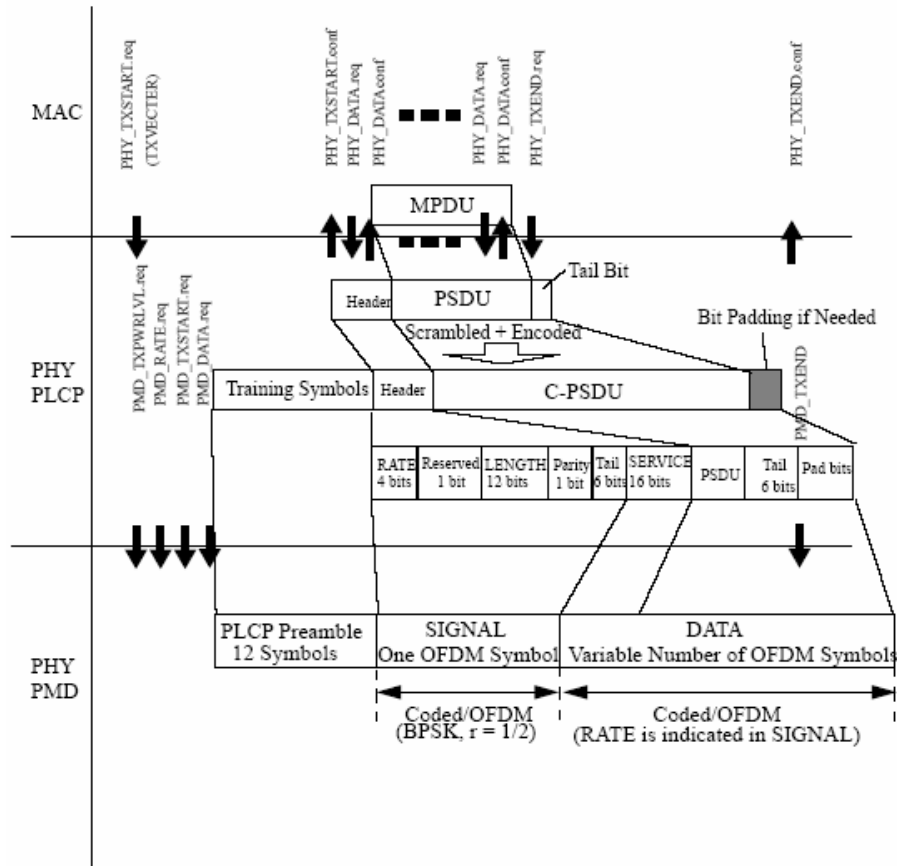


Figure 4. PLCP Transmit Procedure

With 802.11a OFDM modulation, the binary serial signal is divided into groups (symbols) of one, two, four, or six bits, depending on the data rate chosen, and converted into complex numbers representing applicable constellation points. If a data rate of 24 Mbps is chosen, for example, then the PLCP maps the data bits to a 16QAM constellation.

In an OFDM scheme, a large number of orthogonal, overlapping, narrow band subchannels or subcarriers, transmitted in parallel, divide the available transmission bandwidth. The separation of the subcarriers is theoretically minimal such that there is a very compact spectral utilization.

The idea is to distribute the signal over all the carriers and so to spread the information symbols. As a result if a selective carrier fades away it will cause

some error bits in several block symbols and not many error bits in only one symbol. So the channel codification will be able to correct the wrong data by using the correct information that is present in the rest of the symbol thanks to all of the rest frequency carriers which were not fading.

After mapping, the PLCP normalizes the complex numbers in the 802.11a standard to achieve the same average power for all mappings. The PLCP assigns each symbol, having duration of 4 microseconds, to a particular subcarrier. An Inverse Fast Fourier transform (IFFT) combines the subcarriers before transmission.

As with other 802.11 based PHYs, in the 802.11a the PLCP implements a clear channel assessment protocol by reporting a medium busy or clear to the MAC layer via a primitive through the service access point. The MAC layer uses this information to determine whether to issue instructions to actually transmit an MPDU.

3. SIMULINK IMPLEMENTATION

3.1 ABOUT THE SIMULATION

This chapter describes the process for modelling the OFDM PHY layer, the transmission and receiver system, as described in the 802.11a which has been the main source to develop this work. The simulation is intended to be as faithful to the standard layout and its parameters as possible.

The software selected to develop the model has been Matlab and its simulation toolbox Simulink. The OFDM PHY layer system was modelled using Matlab/Simulink to allow various parameters of the system to be varied and tested. The reason to select them is because Matlab is a very common software tool for engineering. Several similar simulations have already been designed with it. Even more, a large number of libraries and toolboxes, mainly in the fields of communications and digital signal processing, are offered by Simulink. The complete model layout is going to be exposed in this chapter, and the files (Wlan80211.m) used to make it work have been added in the Appendix section.

The way to represent the blocks is going to follow the data stream. It is started with the data source and it is ended with the error rate calculation block used to get some results.

Some simulation parameters can be changed in the model in order to view different results; the number of OFDM symbols and the SNR as well can be set in the model. This parameters can be set by clicking in the yellow block called simulation parameters.

This model represents an end-to-end baseband model of the physical layer of a wireless local area network (WLAN) according to the IEEE 802.11a standard. The system supports only a data rate of 54 Mbits/s.

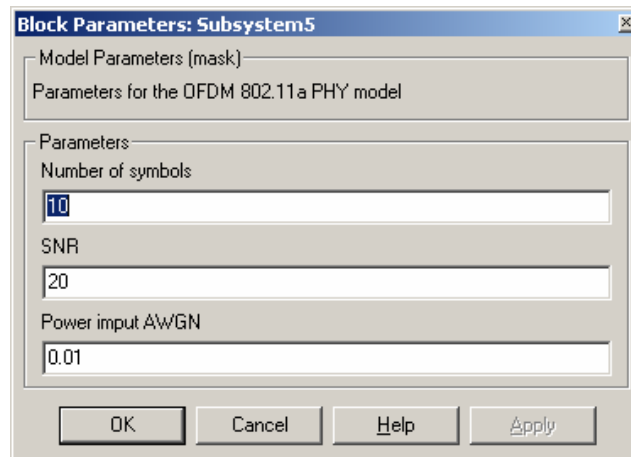


Figure 5. Simulation parameters

Once the model is loaded the parameters for such model are loaded in the workspace. This is achieved by setting the Initialization commands in the simulation parameters block as follow:

```
parameters = Wlan80211a(...  
    SymbolPerFrame, ...  
    SignaltoNoise, ...  
    Inputpwr);  
  
assignin('base', 'parameters', parameters);
```

3.2 STRUCTURE OF THE SYSTEM

The model performs the following tasks:

- Generation of random data at a bit rate of 54Mbps/s.
- Coding/decoding (Viterbi), interleaving/deinterleaving and modulation/demodulation based on the standard's specifications.

This operations are performed under the modulator/demodulator block. The modulator block performs the next tasks:

- Convolutional coding and puncturing using code rate of 3/4
- Data interleaving.
- 64-QAM modulation.

In contrast the demodulator block performs the following tasks:

- 64-QAM demodulation.
 - Data deinterleaving.
 - Viterbi decoding and zero insertion.
-
- OFDM(orthogonal frequency division multiplexing) transmission using 52 subcarriers, 4 pilots, 64-point FFTs, and a 16-sample cyclic prefix.
 - PLCP (physical layer converge protocol) preamble modeled as two long training sequences.
 - AWGN channel.

3.3 OFDM PHY LAYER MODEL USED

The OFDM PHY Layer system was modeled using Simulink and the whole model is shown in the next page (Figure 6). A brief description of the model (blocks and subsystems) is provided afterwards.

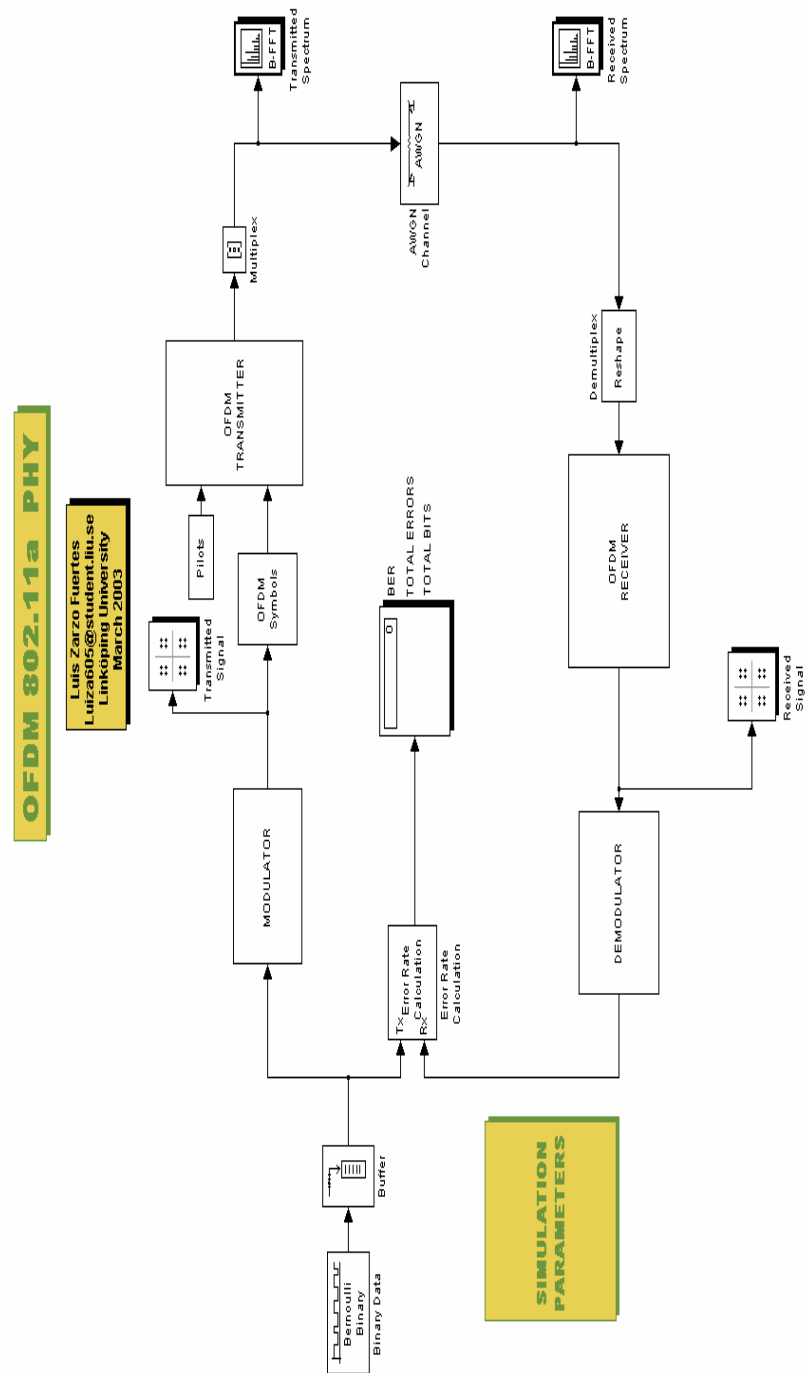


Figure 6. OFDM PHY Model

3.4 MODEL DESCRIPTION

For the model implementation, the parameters for a data rate of 54Mbps/s are considered and the general parameters for a OFDM PHY layer shown in tables 1 and 2 respectively. Those parameters are declared in the file Wlan80211.m shown in the appendix A.

Data rate (Mbps)	Modulation	Coding rate (R)	Coded bits per subcarrier (N_{BPSC})	Coded bits per OFDM symbol (N_{CBPS})	Data bits per OFDM symbol (N_{DBPS})
6	BPSK	$\frac{1}{2}$	1	48	24
9	BPSK	$\frac{3}{4}$	1	48	36
12	QPSK	$\frac{1}{2}$	2	96	48
18	QPSK	$\frac{3}{4}$	2	96	72
24	16QAM	$\frac{1}{2}$	4	192	96
36	16QAM	$\frac{3}{4}$	4	192	144
48	64QAM	$\frac{2}{3}$	6	288	192
54	64QAM	$\frac{3}{4}$	6	288	216

Table 1 Rate-dependent parameters of 802.11a

Parameters	Value
N_{sd} : number of data subcarriers	48
N_{sp} : number of pilot subcarriers	4
N_{fft} : number of FFT subcarriers	64
N_{cyclic} : Cyclic prefix	16
Viterbi Depth : Tradeback depth in the viterbi decoder	34
OFDMSymbolDuration : Duration of one OFDM symbol	4e-006
PunctureVector	[1 1 0 1 1 0]

Table 2 Global parameters

3.4.1 Binary Data Generation

The data is random generated with a Bernoulli Binary Generator block. 216 samples per time are used because, based on the standard and for a 54 Mbits/s data rate, this is the number of samples needed.

This block is in charge of the random data generation at a bit rate of 54 Mbits/s. As shown in table number 1, the number of Ndbps for such data rate is 216. In table number 2 can be seen that the OFDM symbol duration is 4 microseconds. The set parameters are set according to these values.

Binary data is created and stored in a buffer. The output of that buffer depends on the number of OFDM symbols per frame chosen in the simulation parameters.

3.4.2 TRANSMISSION SYSTEM

3.4.2.1 MODULATOR

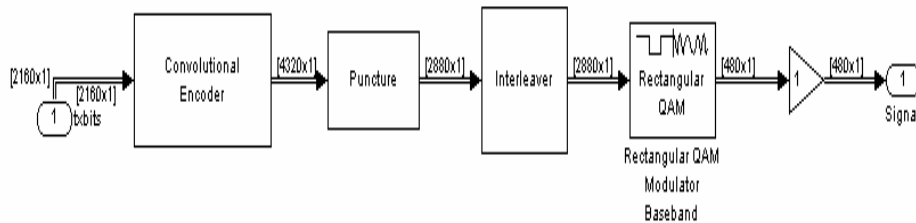


Figure 7. Modulator subsystem

The modulator subsystem is in charge of the following tasks:

- Convolutional coding and puncturing using 3/4 code rate
- Data interleaving
- 64-QAM modulation.

3.4.2.1.1 Convolutional coding

Error control coding, or channel coding, is a method of adding redundancy to the sent information so that it can be transmitted over a noisy channel, and subsequently be checked and corrected for errors that occurred in the transmission. Convolutional codes are commonly specified by three parameters n , k and m , where n is the number of output bits, k is the number of input bits and m is the number of memory registers.[23]

The convolutional encoder is used to correct random error. The coding rate is $1/2$ and the constraint length is 7. Since different data rates are supported in 802.11a OFDM PHY, puncture is needed after Convolutional coding.

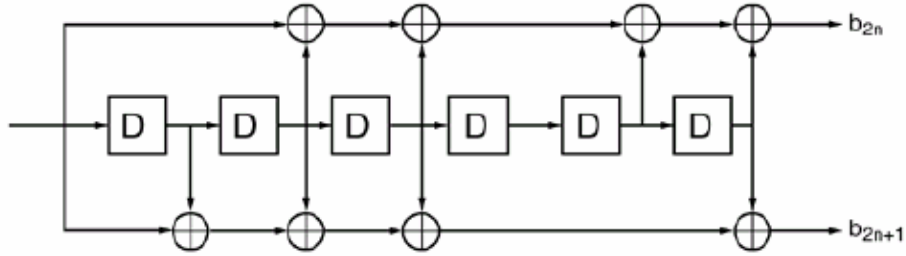


Figure 8. Convolutional encoder(K=7)

3.4.2.1.2 Puncturing Convolutional Codes

Puncturing is a very useful technique to generate additional rates from a single convolutional code. The basic idea behind puncturing is to not transmit some of the output bits from the convolutional encoder, thus increasing the rate of the code. This increase in rate decreases the free distance of the code, but usually the resulting free distance is very close to the optimum one that is achieved by specifically designing a convolutional code for the punctured rate. The receiver inserts dummy bits to replace the punctured bits in the receiver, hence only one encoder/decoder pair is needed to generate several different code rates.

Code puncturing involves not transmitting certain code bits. The encoder for a punctured code can be fabricated using the original low-rate convolutional encoder followed by a bit selector which deletes specific code bits according to a given puncturing rule. The Puncture block carries out the puncturing, periodically removing bits from the encoded bit stream, thereby increasing the code rate.

The puncture pattern is specified by the Puncture vector parameter in the mask. The puncture vector is a binary column vector. A 1 indicates that the bit in the corresponding position of the input vector is sent to the output vector, while a 0 indicates that the bit is removed.

To create a rate $3/4$ code from the rate $1/2$, constraint length 7 convolutional code, the optimal puncture vector is $[1\ 1\ 0\ 1\ 1\ 0]^T$ (where the T after the vector indicates the transpose). Bits in positions 1, 2, 4, and 5 are transmitted, while bits in positions 3 and 6 are removed. Now, for every 3 bits of input, the punctured code generates 4 bits of output (as opposed to the 6 bits produced before puncturing). This makes the rate $3/4$.

3.4.2.1.3 Data interleaving

Interleaving aims to distribute transmitted bits in time or frequency or both to achieve desirable bit error distribution after demodulation.

All encoded data bits shall be interleaved by a block interleaver with a block size corresponding to the number of bits in a single OFDM symbol, $N_{CBPS}(288)$. The interleaver is defined by a two-step permutation. The first permutation ensures that adjacent coded bits are mapped onto nonadjacent subcarriers. The second ensures that adjacent coded bits are mapped alternately onto less and more significant bits of the constellation and, thereby, long runs of low reliability (LSB) bits are avoided.[22]

Deinterleaving is the opposite operation of interleaving; i.e., the bits are put back into the original order.

3.4.2.1.4 64-QAM modulation

Modulation is the process by which information signals, analog or digital, are transformed into waveforms suitable for transmission across the channel. Hence, digital modulation is the process by which digital information is transformed into digital waveforms.

After bit interleaving, the data bits are entered serially to the constellation mapper. The data bits shall be modulated by using 64-QAM modulation; the encoded and interleaved binary serial input data shall be divided into groups of NBPS (6) bits and converted into complex numbers representing 64-QAM constellation points. The conversion shall be performed according to Gray-coded constellation mappings,[22] as illustrated in the next figure.

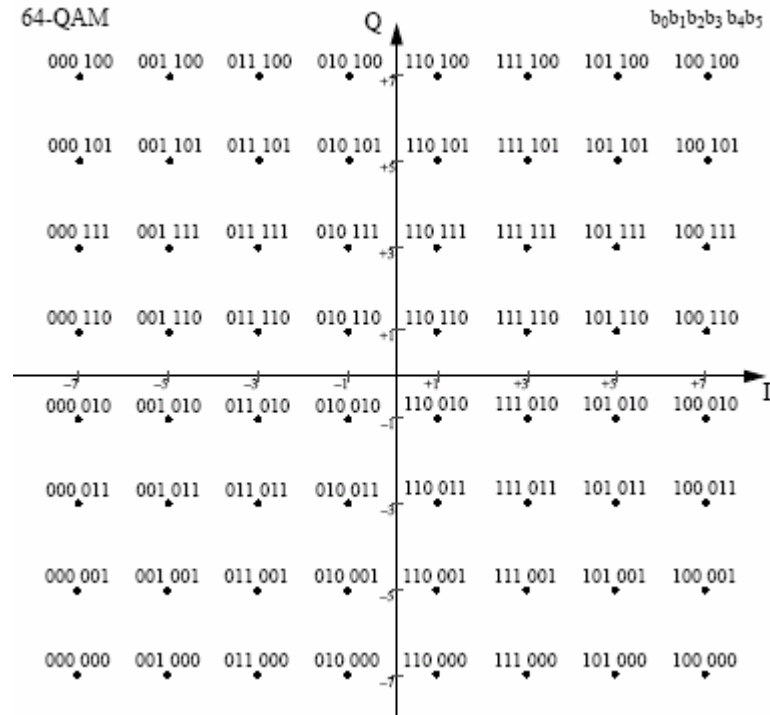


Figure 9. 64-QAM constellation points

After the modulation the constellation diagram (in the model) looks like this:

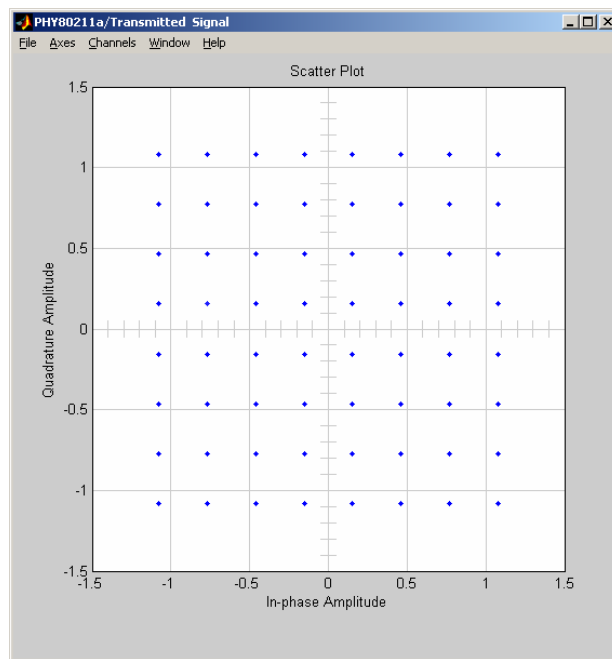


Figure 10. Constellation diagram after modulation

3.4.2.2 OFDM symbols

The serial signal is transformed to parallel after the modulation using a reshape block.

3.4.2.3 OFDM TRANSMITTER

OFDM (orthogonal frequency division multiplexing) transmission uses 52 subcarriers, 4 pilots, 64-point FFTs, and a 16-sample cyclic prefix. The figure illustrates the OFDM transmission.

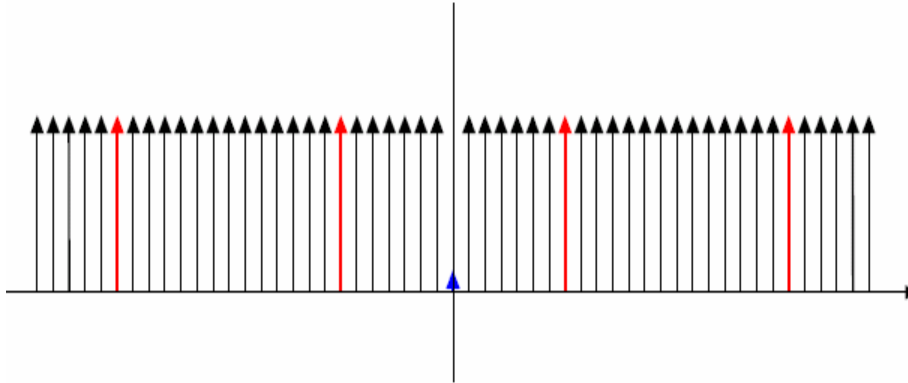


Figure 11. 52 subcarriers, 4 of them pilots

The OFDM Transmitter subsystem performs the following task:

- Pilots and preamble insertion
- IFFT
- Cyclic prefix addition

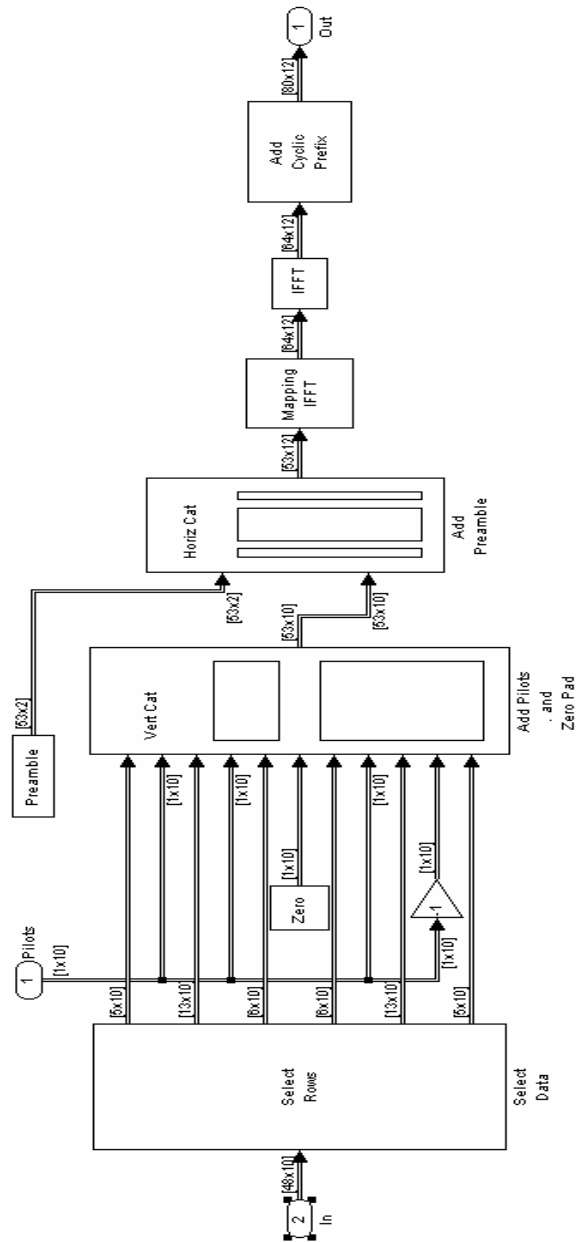


Figure 12. OFDM Transmitter subsystem

3.4.2.3.1 Pilot insertion

In each OFDM symbol, four of the subcarriers are dedicated to pilot signals in order to make the coherent detection robust against frequency offsets and phase noise. These pilot signals shall be put in subcarriers -21, -7, 7 and 21.[22] Illustrated in the next figure.

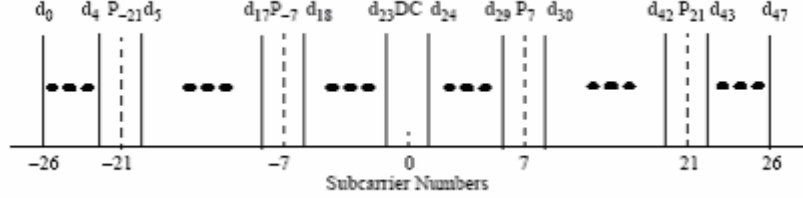


Figure 13. Subcarrier frequency allocation

As explained, four pilots are inserted in each OFDM symbol. Pilots are generated by a polynomial generator:

$$x^7 + x^4 + 1$$

In the model the pilots are generated by a PN sequence generator block with the mentioned generator polynomial. Four pilots are inserted in each OFDM symbol in the required subcarriers. The insertion is achieved with a matrix concatenation block, pilots are inserted in its proper place in each symbol.

3.4.2.3.2 Preamble

The preamble is used to detect the start of the packet and to synchronize the receiver as well. The OFDM symbols should be packed into frames before being sent. A preamble is added at the beginning of each frame.[23] It helps the receiver to estimate phase and amplitude errors, thereby allowing it to correct the received signal.

In the simulation exposed in this work a preamble consisting on two long training symbols, like the following one, has been used:

$$L_{[-26,25]} = \{1, 1, -1, -1, 1, 1, -1, 1, -1, 1, 1, 1, 1, 1, 1, -1, -1, 1, 1, -1, 1, -1, 1, 1, 1, 1, 0, 1, -1, -1, 1, 1, -1, 1, -1, 1, -1, -1, -1, -1, -1, 1, 1, -1, -1, 1, -1, 1, 1, -1, 1, 1, 1\}$$

A long OFDM training symbol consists of 53 subcarriers (including a zero value at DC).

3.4.2.3.3 IFFT/FFT

The most common way to implement the inverse Fourier transform is by an inverse Fast Fourier Transform (IFFT) algorithm. A 64-point IFFT is used; the coefficients 1 to 26 are mapped to the same numbered IFFT inputs, while the coefficients -26 to -1 are copied into IFFT inputs 38 to 63. The rest of the inputs, 27 to 37 and the 0 (dc) input, are set to zero. This mapping is illustrated in Figure 15. After performing an IFFT, the output is cyclically extended to the desired length.[22]

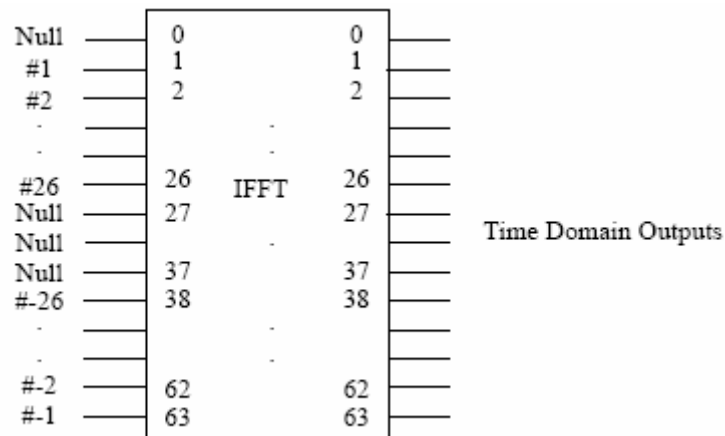


Figure 14. FFT Mapping

Zero pad and selector blocks are used for this assignment. Zero pad appends zeros to the input signal and the selector block reorders the subcarriers as shown in the previous figure.

An inverse Fourier transform converts the frequency domain data set into samples of the corresponding time domain representation of this data. Specifically, the IFFT is useful for OFDM because it generates samples of a waveform with frequency components satisfying orthogonality conditions. Then, the parallel to serial block creates the OFDM signal by sequentially outputting the time domain samples.

The receiver performs the inverse of the transmitter. First, the OFDM data are split from a serial stream into parallel sets. The Fast Fourier Transform (FFT) converts the time domain samples back into a frequency domain representation. Finally, the parallel to serial block converts this parallel data into a serial stream to recover the original input data. The following illustration shows the IFFT/FFT.

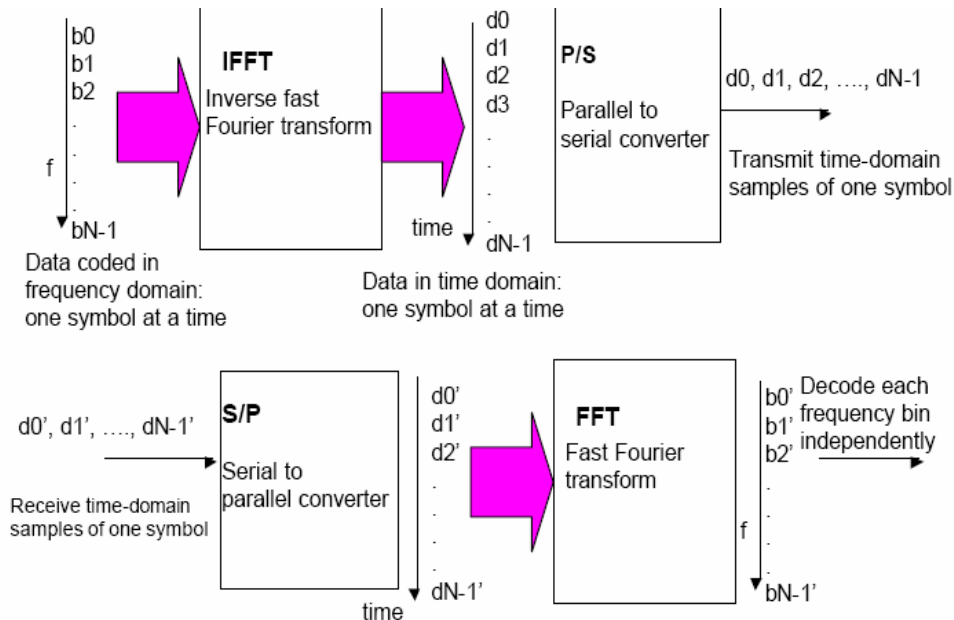


Figure 15. IFFT/FFT description

IFFT/FFT operation ensures that sub-carriers do not interfere with one another. Since the sub-carriers do not interfere, they can be brought closer together. Orthogonality between the sub-carriers allows their overlapping while avoiding the occurrence of crosstalks. Thus, a significant bandwidth saving can be achieved by using an orthogonal multicarrier technique, as it can be seen in the next figure.

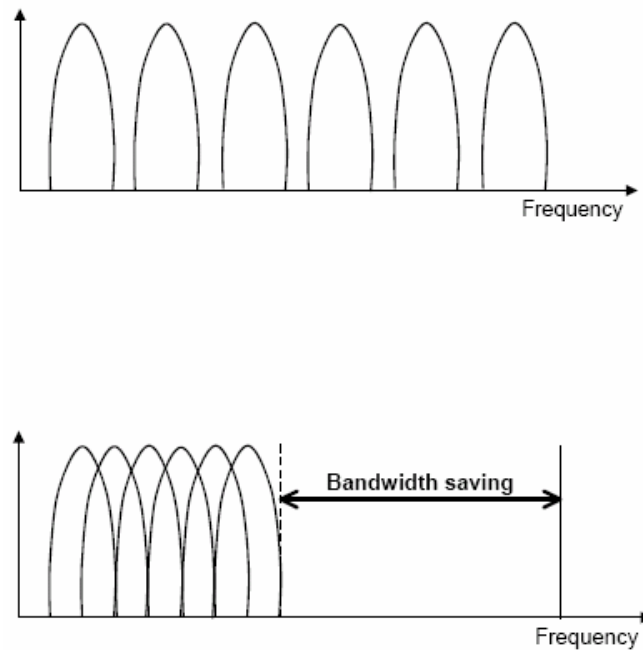


Figure 16. Orthogonal multicarrier technique

3.4.2.3.4 Cyclic prefix addition

A cyclic prefix is added to avoid problems in the receiver caused by the mixing of subsequent symbols in the receiver. One way for avoiding this mix between symbols is by extending the symbol (cyclic prefix or guard interval). The cyclic prefix preserves orthogonality between sub-carriers.

The cyclic prefix is formed by a cyclic continuation of the signal so that the information in the cyclic prefix is actually present in the OFDM symbol. The next figure shows clearly the purpose of using a cyclic prefix.

If two OFDM symbols interfere with each other it is just because the cyclic prefix and part of the data field of one of them has been overlapped by the other one. This can happen when the transmission delay of the symbols is bigger than the cyclic prefix duration. To avoid the ISI, a collection of the inter symbol interference caused by all elements of the system and that can reduce the effective data rate of wireless LAN transceivers, the system has to guarantee that no symbol echoes longer than the duration of the cyclic prefix. The way to

secure that without increasing the cyclic prefix is by establishing a maximum delay path for the signal.

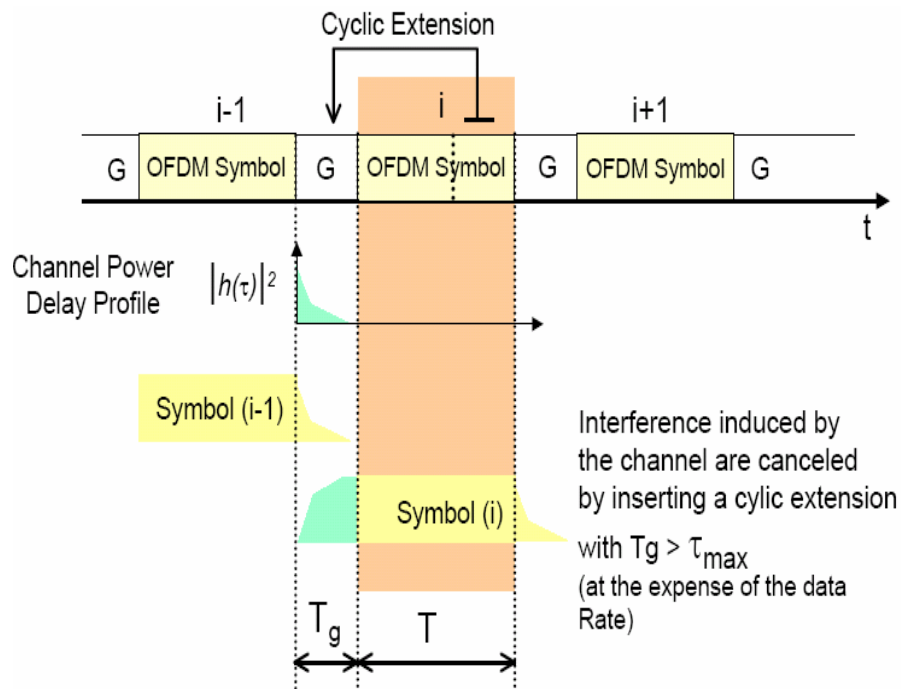


Figure 17. Cyclic prefix

In the previous illustrations the purpose of adding a cyclic prefix at the beginning of each symbol can be clearly seen. A selector block is used for implementing this in the model; the last 16 subcarriers are copied into the beginning of the OFDM symbols.

3.4.2.4 Multiplex

The multiplex block is used to convert the signal from parallel to serial and to transmit time-domain samples of one symbol.

3.4.2.5 AWGN

The AWGN Channel block adds white Gaussian noise to a real or complex input signal. When the input signal is real, this block adds real Gaussian noise and produces a real output signal. When the input signal is complex, this block adds complex Gaussian noise and produces a complex output signal. This block inherits its sample time from the input signal.[7]

3.4.3 RECEIVER SYSTEM

3.4.3.1 Demultiplex

The signal is converted from serial to parallel using a reshape block.

3.4.3.2 OFDM receiver

The OFDM receiver subsystem performs the following tasks:

- Remove Cyclic prefix added in the transmission side.
- FFT
- Remove the zeros added for the FFT and reorder the subcarriers
- Remove the preamble
- Remove the pilots.

The blocks used for performing those tasks are selector blocks and the FFT block.

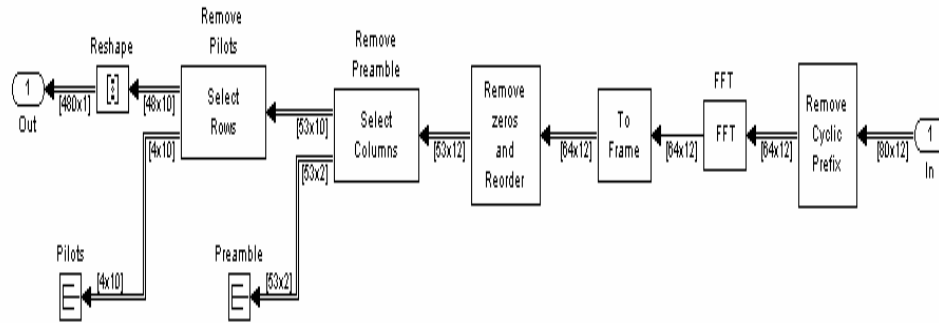


Figure 18. OFDM receiver

3.4.3.3 Demodulator

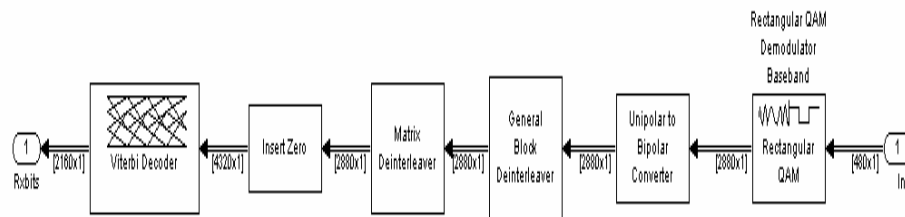


Figure 19. Demodulator

The demodulator subsystem performs the following tasks (in fact, it performs the inverse tasks of the modulator subsystem):

- Demodulation of the signal
- Deinterleaver
- Zero insertion (opposite of the puncturing)
- Viterbi decoding

3.4.3.3.1 Viterbi decoding

A receiver viterbi decoder is used to decode the convolutional codes with a trace back depth of 34. Viterbi decoding is the best known implementation of the maximum likely-hood of convolutional codes. The principal use to reduce the choices is that the errors occur infrequently so that the probability of an error is small and the probability of two errors in a row is much smaller than a single error, that is because the errors are distributed randomly. The Viterbi decoder examines an entire received sequence of a given length. The decoder computes a metric for each path and makes a decision based on this metric. All paths are followed until two paths converge on one node. Then the path with the higher metric is kept and the one with lower metric is discarded. The paths selected are called the survivors. For an N bit sequence, the total number of possible received sequences is 2 to the power of N. The most common metric used is the Hamming distance metric, Hard Decoding. This is just the dot product between the received codeword and the allowable codeword.

A Viterbi decoder based on the mother code decoder is used for decoding the punctured codes of the family. To decode different rate codes, only metrics are changed according to the same puncturing rule used by the encoder (the deleted bits are not counted when calculating the path metrics).

3.4.4 ERROR RATE CALCULATION

The Error Rate Calculation block compares input data from a transmitter with input data from a receiver. It calculates the error rate as a running statistic by dividing the total number of unequal pairs of data elements by the total number of input data elements from one source.[7]

One can use this block to compute either symbol or bit error rate because it does not consider the magnitude of the difference between input data elements. If the inputs are bits, the block computes the bit error rate. If the inputs are symbols, it computes the symbol error rate.

The input ports marked Tx and Rx accept transmitted and received signals respectively. The Tx and Rx signals must share the same sampling rate.

This block produces a vector of length three whose entries correspond to:

- The error rate (BER- bit error rate)
- The total number of errors, i.e., the comparisons between unequal elements
- The total number of comparisons that the block made

3.5 SIMULATION RESULTS AND ANALYSIS

3.5.1 Gaussian Noise Tolerance of 802.11a OFDM PHY

Once finished the design and implementation of the OFDM PHY layer using Simulink, it is time to perform some analysis and comment the obtained results.

The next figures show the results of the simulations. In them, the difference of the received signals due to the SNR(Signal to Noise Ratio) value set can be seen. The lower the ratio between the signal and the noise, the more is the received signal affected by the noise. As a result of this the BER(Bit error rate) increases. The number of OFDM symbols in all the simulations was fixed to 20.

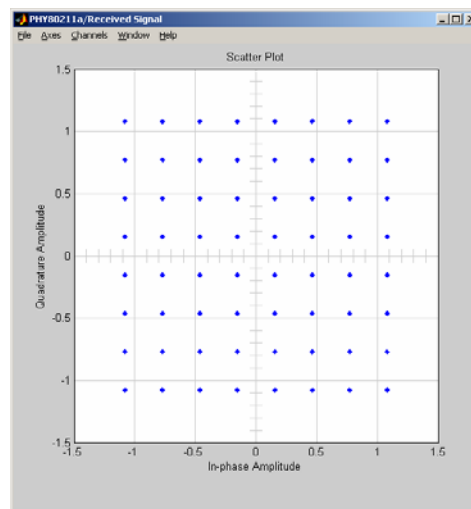


Figure 20. Received signal SNR = 50

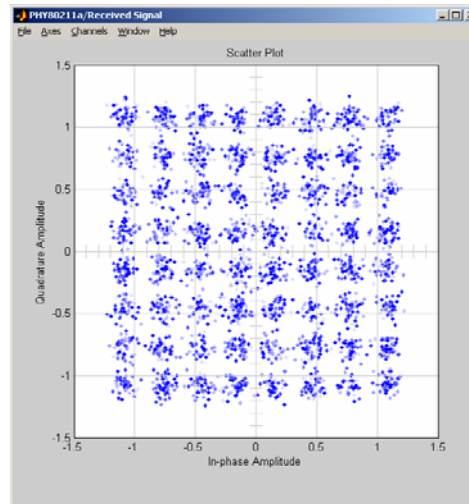


Figure 21. Received signal SNR = 20

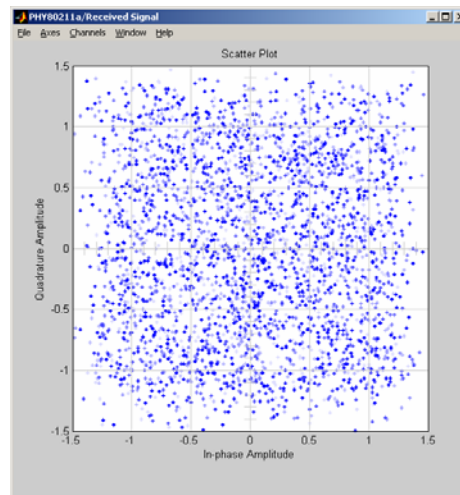


Figure 22. Received signal SNR = 10

The results show that with a data rate of 54 Mbits/s the transmission can tolerate a SNR of approximately $> 20\text{dB}$. The bit error rate BER deteriorates more rapidly as the SNR drops below 16dB, as it can be seen in the next graphic.

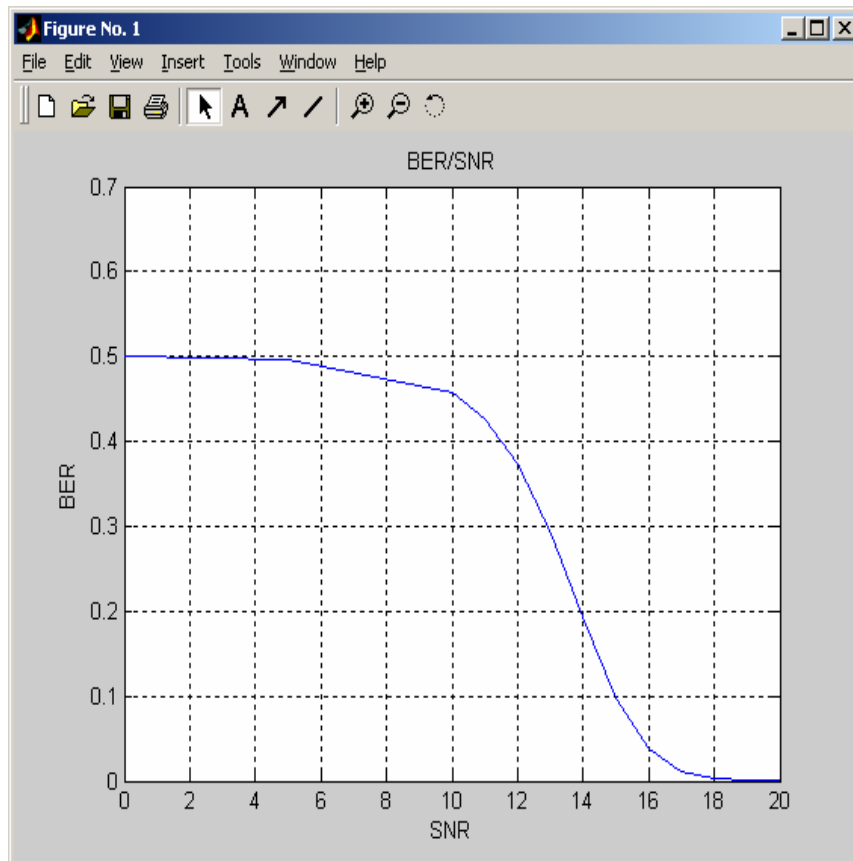


Figure 23. BER/SNR Graphic

4. DIGITAL SIGNAL PROCESSING

4.1 INTRODUCTION TO DSP

Digital Signal processing (DSP) is one of the fastest growing fields of technology and computer science in the world. In today's world almost everyone uses DSPs in their everyday life but, unlike PC users, almost no one knows that he/she is using DSPs. Digital Signal Processors are special purpose microprocessors used in all kind of electronic products, from mobile phones, modems and CD players to the automotive industry; medical imaging systems to the electronic battlefield and from dishwashers to satellites.[17]

DSP is all about analysing and processing real-world or analogue signals, i.e. the kind of signals that humans interact with, for example speech. These signals are converted to a format that computers can understand (digital) and, once this has happened, process. The following diagram shows the typical component parts of a DSP system.

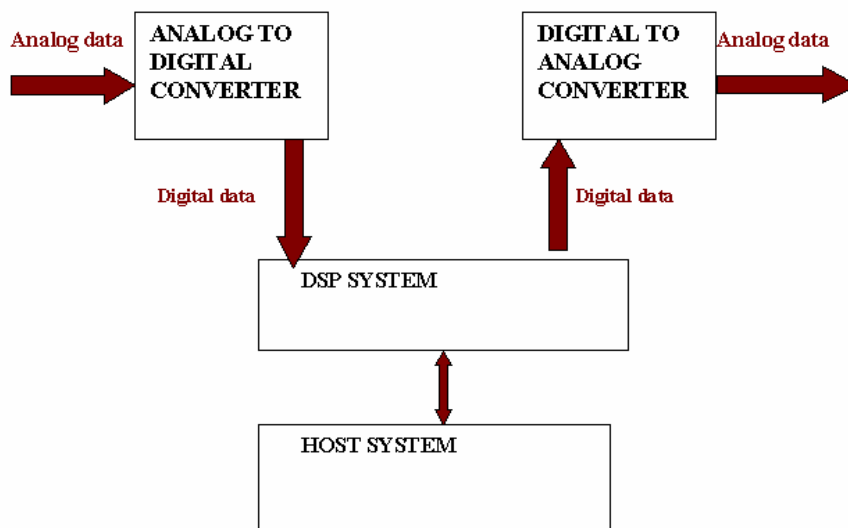


Figure 24. Typical components of a DSP system

In order to process analog signals with digital computers they must first be converted to digital signals using analog to digital converters. Similarly, the

digital signals must be converted back to analog ones for them to be used outside the computer.

There are many reasons why we process these analog signals in the digital world. Traditional signal processing was achieved by using analogue components such as resistors, capacitors and inductors. However, the inherent tolerance associated with this components, temperature and voltage changes and mechanical vibrations can dramatically affect the effectiveness of analogue circuitry. On the other hand, DSP is inherently stable, reliable and repeatable. With DSP it is easy to change, correct or update applications. Additionally, DSP reduces noise susceptibility, chip count, development time, cost and power consumption.

4.2 HOW DSPs ARE DIFFERENT FROM OTHER MICROPROCESSORS

DSP has many unique properties. It is a Super Mathematician thanks to its arithmetic logic units and its optimized multipliers. DSPs do really well in application where the data to be processed is arriving in a continuous flow, often referred to as a stream. It uses almost no power compared to a PC microprocessor.

Next, some features that make DSP different from other microprocessors are going to be described:

- High speed arithmetic: Most DSP operations require additions and multiplications together. DSP processors usually have hardware adders and multipliers which can be used in parallel within a single instruction, so both, an addition and a multiplication, can be executed in a single cycle. Thus, DSP processors arithmetic speed is very high compared with microprocessors.
- Data transfer to and from real world. In a typical DSP application the processor will have to deal with multiple sources of data from the real world. In each case, the processor may have to be able to receive and transmit data in real time, without interrupting its internal mathematical operations. These multiple communications routes mark the most important distinctions between DSP processors and general purpose processors.

- Multiple access memory architectures: Typical DSP operations require many simple additions and multiplications. To fetch the two operations in a single instruction cycle the two memory accesses should be able to operate simultaneously. For this reason DSP processors usually support multiple memory accesses in the same instruction cycle.
- Digital Signal Processors also have the advantage of consuming less power and being relatively cheap.

4.3 INTRODUCTION TO THE TMS320C6000 PLATFORM OF DIGITAL SIGNAL PROCESSORS

The DSP architecture is a well defined but quite complex hardware structure that needs much time to be explained in detail. An overview of this architecture is going to be exposed here in order to make it as much understandable as possible.

The TMS320C6000 family of processors from the company Texas Instruments is designed to meet the real-time requirements of high performance digital signal processing. With a performance of up to 2000 million instructions per second (MIPS) at 250 MHz and a complete set of development tools, the TMS320C6000 DSPs offer cost effective solutions to higher-performance DSP programming challenges.

The TMS320C6000 DSPs give the system architects unlimited possibilities to differentiate their products. High performance, easy use, and affordable pricing make the TMS320c6000 platform the ideal solution for a large number of applications (multichannel multifunction applications such as: pooled modems, wireless local loop base stations, multichannel telephony systems, etc).

First of all, a DSP device must be considered as a specific microprocessor whose components have been linked in a clever way to process faster. The TMS320C6xxx family are processors currently running at a clock speed of up to 300MHz (225MHz in the TMS320C6713 case). The C62xx processors are fixed-point processors whereas the C67xx are floating-point processors. These refer to the format used to store and manipulate numbers withing the devices.

Figure 24 shows the main components of the TMS320C6000 DSP under a

block diagram form. It is composed of:

- External Memory Interface (EMIF) to access external data at the specified address.
- Memory, which is the internal memory where a set of instructions and data values can be stored (FFT algorithm for example).
- Peripherals are the possible connectable devices that can be associated with the DSP (DMA/EDMA, Serial port, Timer/Counter...).
- Internal buses; they allow the components to quickly communicate together differentiating addresses and data.
- CPU, which is the most important component since it performs all the operation.

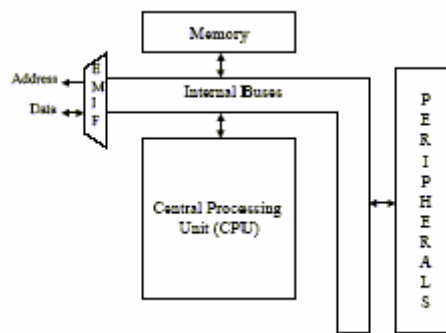


Figure 25.TMS320C6000 block diagram

4.3.1 TMS320C6713 DSP Description

The TMS320C67 DSPs (including the TMS320C6713 device) compose the floating-point DSP generation in the TMS320C6000 DSP platform. The TMS320C6713 (C6713) device is based on the high-performance, advanced VelociTI very-long-instruction-word (VLIW) architecture developed by Texas Instruments (TI), making this DSP an excellent choice for multichannel and multifunction applications.

The DSK features the TMS320C6713 DSP, a 225 MHz device delivering up to 1800 million instructions per second (MIPs) and 1350 MFLOPS. This DSP generation is designed for applications that require high precision accuracy. The C6713 is based on the TMS320C6000 DSP platform designed to fit the needs of high-performing high-precision applications such as pro-audio, medical and diagnostic. Other hardware features of the TMS320C6713 DSK board include:

- Embedded JTAG support via USB
- High-quality 24-bit stereo codec
- Four 3.5mm audio jacks for microphone, line in, speaker and line out
- 512K words of Flash and 8 MB SDRAM
- Expansion port connector for plug-in modules
- On-board standard IEEE JTAG interface
- +5V universal power supply

The DSP environment used in this project is the TMS320C6713 DSK. Figure 25 shows the architecture of the TMS320C6713 DSK. Key features include:

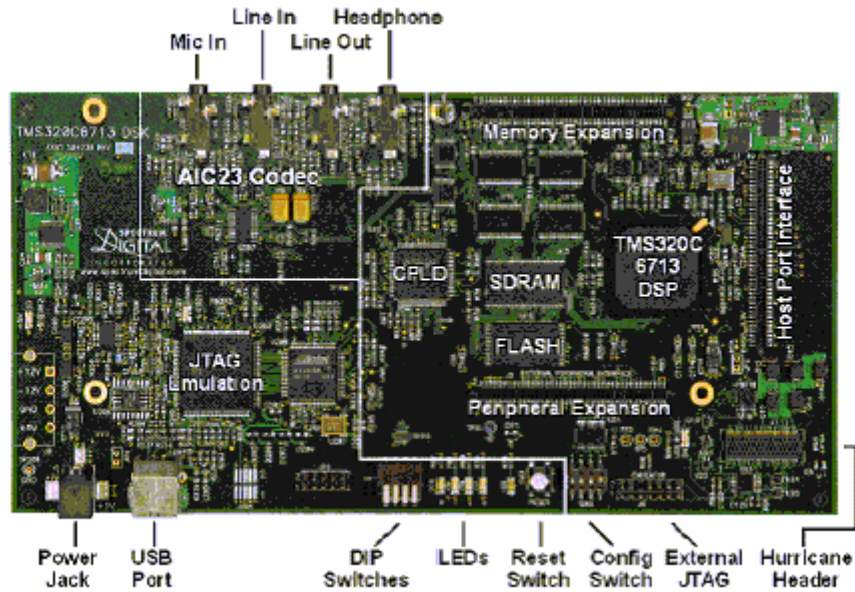


Figure 26. Architecture of the TMS320C6713 DSK

4.4 DSP IMPLEMENTATION

Once the implementation of the model in simulink is achieved ,and already tested that it works properly, the next step is targeting the model on the dsk board.

In this section, the implementation of the model on a TMS320C6713 DSP board from Texas Instrument is carried out. Having the implementation of the model in Simulink and after testing that this is working properly, the next step is targeting the model on the dsp board. For accomplishing this assignment, the following tools have been used:

- Code Composer Studio v 2.21
- Embedded Target for TI c6000 DSP

A brief description of these tools is provided next.

4.4.1 The Code Composer Studio

Code Composer Studio extends the basic code generation tools with a set of debugging and real-time analysis capabilities. Code Composer Studio supports all phases of the development cycle shown here.

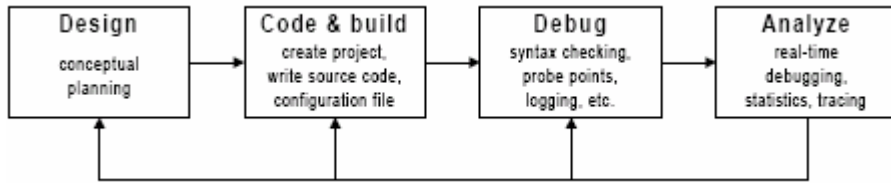


Figure 27. Development cycle

In the following figure can be seen how the Code Composer Studio componets work together:

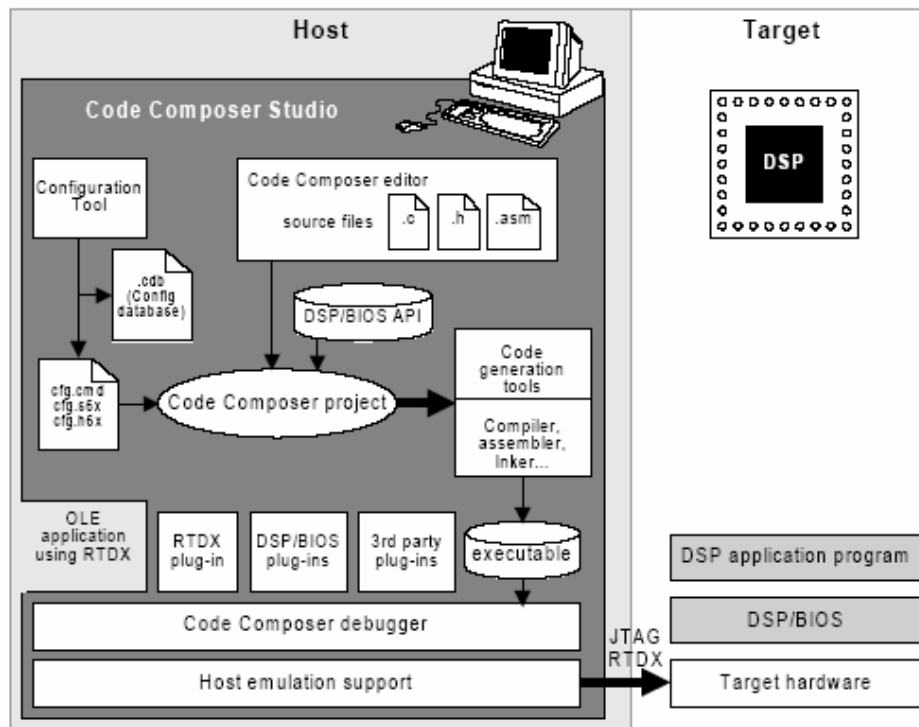


Figure 28. CCS Components

Code composer works with a project paradigm. Essentially, within CCS you must create a project for each executable program you wish to create. A project stores all the basic information to build the executable file ("project".out). The project structure is shown in the project window, as can be seen in the next figure:

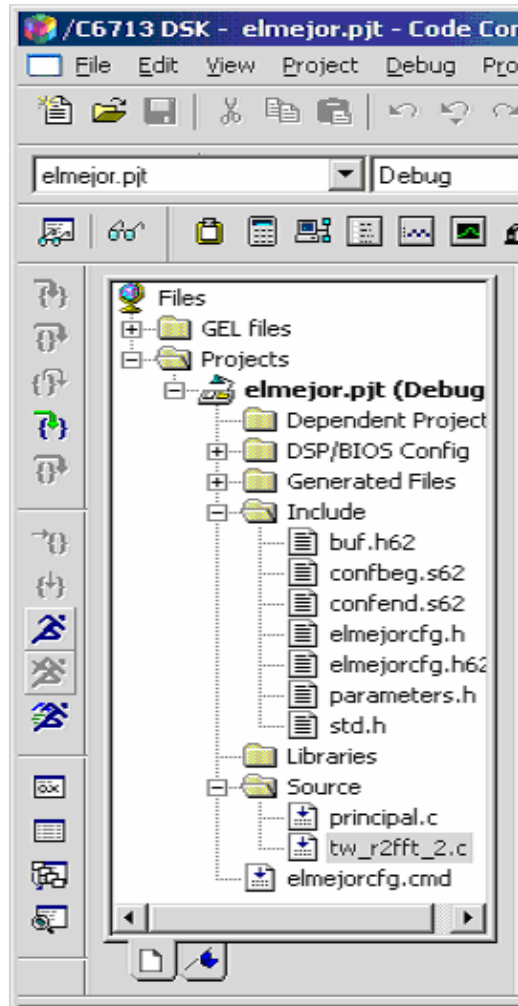


Figure 29. Project structure

In a project the following kind of files can be found:

- **Include files:** Typical header files used in the application development in C language. Is not necessary to add those kind of files in the project, on the contrary is the CCS itself the one in charge of loading those files into the project.
- **Library files:** User's library files are included in this folder
- **Source files:** The different modules that compose the application are included in this folder. They can be developed in C language (.c) or in assembly language(.asm).
- **Cmd files:** Files used by the linker in order to describe the way in which the different sections built in the application are located in the different memory address the platform dispose form running the current application.

4.4.2 Embedded Target for TI c6000 DSP.

For implementing and running the OFDM PHY layer model on the DSP board, a MATLAB tool called Embedded Target for TI C6000 DSP has been used.

4.4.2.1 What is Embedded Target for the TI TMS320C6000 DSP Platform

Embedded Target for the TI TMS320C6000 DSP Platform integrates Simulink® and MATLAB® with Texas Instruments eXpressDSP™ tools. The software collection lets you develop and validate digital signal processing designs from concept through code. The Embedded Target for TI C6000 DSP consists of the TI C6000 target that automates rapid prototyping on your C6000 hardware targets. The target uses C code generated by Real-Time Workshop® and your TI development tools to build an executable file for your targeted processor. The Real-Time Workshop build process loads the targeted machine code to your board and runs the executable file on the digital signal processor.[3]

Using the Embedded Target for TI C6000 DSP and Real-Time Workshop, you can create executable code for the following boards:

- C6416 DSP Starter Kit from Texas Instruments
- C6701 Evaluation Module from Texas Instruments
- C6711 DSP Starter Kit from Texas Instruments
- C6713 DSP Starter Kit from Texas Instruments (board used in this thesis work)***
- TMDX326040. A daughter card for the C6711 DSK

4.4.2.2 About Embedded Target for C6000 DSP

Embedded Target for TI C6000 DSP lets you use Simulink to model digital signal processing algorithms from blocks in the DSP Blockset or in another Simulink library, and then use Real-Time Workshop to generate ANSI C code targeted to the Texas Instruments DSP development boards. The Embedded Target for TI C6000 DSP takes the generated C code and uses Texas Instruments (TI) tools to build specific machine code depending on the TI board you use. The build process downloads the targeted machine code to the selected hardware and runs the executable on the digital signal processor. After downloading the code to the board, your digital signal processing (DSP) application runs automatically on your target.[3]

4.4.2.3. Operation of embedded target for C6000 DSP

In this section a brief operation overview of the Embedded Target is given. Also some indications about how the tool should be configured for working in a proper way are provided as well.

With the model open, select Simulation Parameters from the Simulink option to open the Simulink Parameters dialog box. From this dialog, click Real-Time Workshop. In the Real-Time Workshop panel of the dialog, specify the correct target file by clicking Browse and selecting ti_c6000.tlc from the list of targets. After selecting the target configuration, Real-Time Workshop chooses the appropriate system target file, template makefile, and make command. The target configuration filename can also be entered and Real-Time Workshop will fill in the template makefile and make command selections.

Real-Time Workshop uses template makefiles to generate the makefile for building the executable file. During the automatic build process, the make_rtw command is issued. Make_rtw extracts information from the template makefile ti_c6000.tmf and creates the actual makefile c6000.mk. When Real-Time

Workshop compiles the model, it uses the actual makefile to generate the compiled code for the target. Set the Template makefile option to `ti_c6000.tmf` when the application for the C6000 target is built. If the template makefile shown in the option is not `ti_c6000.tmf`, click Browse to open the list of available system target files and select the correct file from the list. Real-Time Workshop then selects the appropriate template makefile.[3]

To generate code from the digital signal processing application, use the standard command `make_rtw` as the Make command. On the Real-Time Workshop panel of the Simulation Parameters dialog, under Configuration in the Target configuration category, enter `make_rtw` for the Make command option. Parameters set in this dialog belong to the model that is being building. They are saved with the model and stored in the model file. The option Generate code only does not apply to the Embedded Target for TI C6000 DSP targeting. To configure Embedded Target for TI C6000 DSP to generate source code without building and executing the code on the target and without requiring Code Composer Studio, select Generate code only for the Build action under the TI C6000 runtime options.

With this configuration, a real-time executable can be generated and be downloaded to the TI development boards. Doing this by clicking Build on the Real-Time Workshop panel. Real-Time Workshop automatically generates C code. During the same build operation, the template makefile and block parameter dialog entries get combined to form the target makefile for the TI C6000 board. The makefile invokes the TI cross-compiler to build an executable file. If the Build and execute build action is selected, the executable file is automatically downloaded via the peripheral component interface (PCI) or over the parallel port to the C6711 DSK. After downloading the executable file to the target, the build process runs the file on the board's DSP.

4.4.2.4 GETTING STARTED WITH EMBEDDED TARGET

Once installed the software in the computer (a trial version is installed, not the current one, and old one from 2003) and familiarized ourselves about the use of this software thanks to the manual, we are ready to generate C code from the Simulink model using Real-Time Workshop/Embedded target for targeting on to the DSP board.

The Simulink model is loaded , and, as explained before, the first step is to specify the correct target file (`ti_c6000.tlc`). Simulation Parameters should be selected from the Simulink option to open the Simulink Parameters dialog box.

On this dialog box, click on Real-Time Workshop. In the Real-Time Workshop panel of the dialog box, specify the correct target file, clicking Browse and selecting `ti_c6000.tlc` from the list of targets.

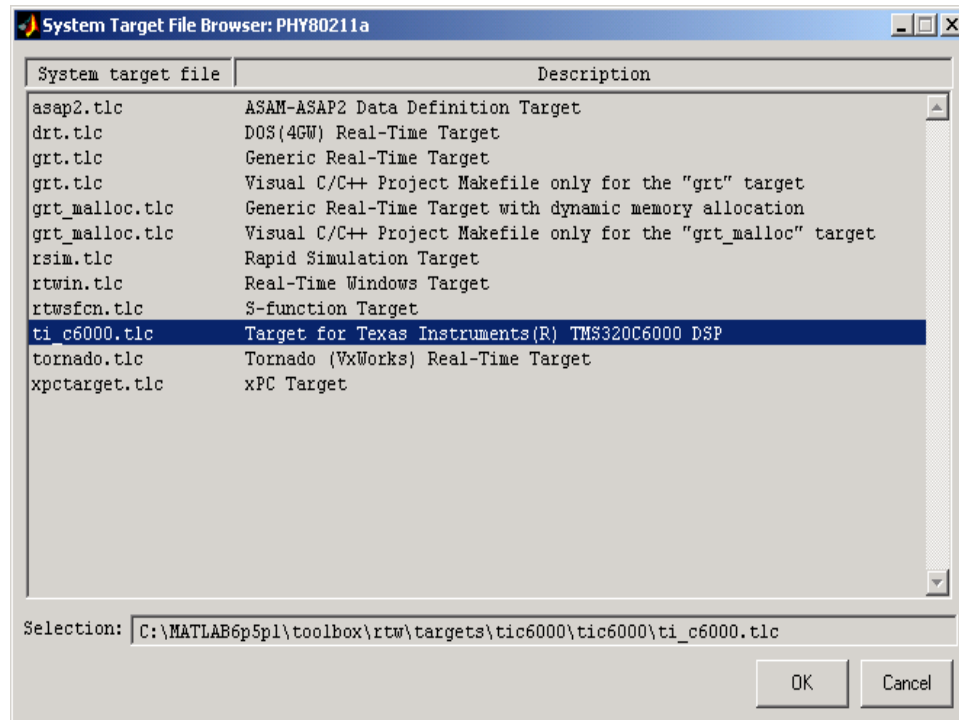


Figure 30. System Target File Browser

Once selected the system target file `ti_c6000.tlc` and clicked OK for confirmation, we face the first error, shown by the next warning message.

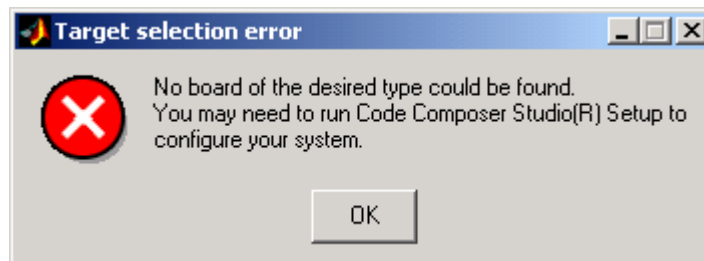


Figure 31. Target error

The message does not say that no board was found but that no board of the

desired type could be found. The cause could be that the board is not compatible with this software, but referring to the documentation, can be seen that the board used is included into the supported ones.

The warning message advise to run Code Composer Studio Setup to configure the system. That task was already done and the board was already configured in the system, as the next figure, the CCS Setup window, shows.

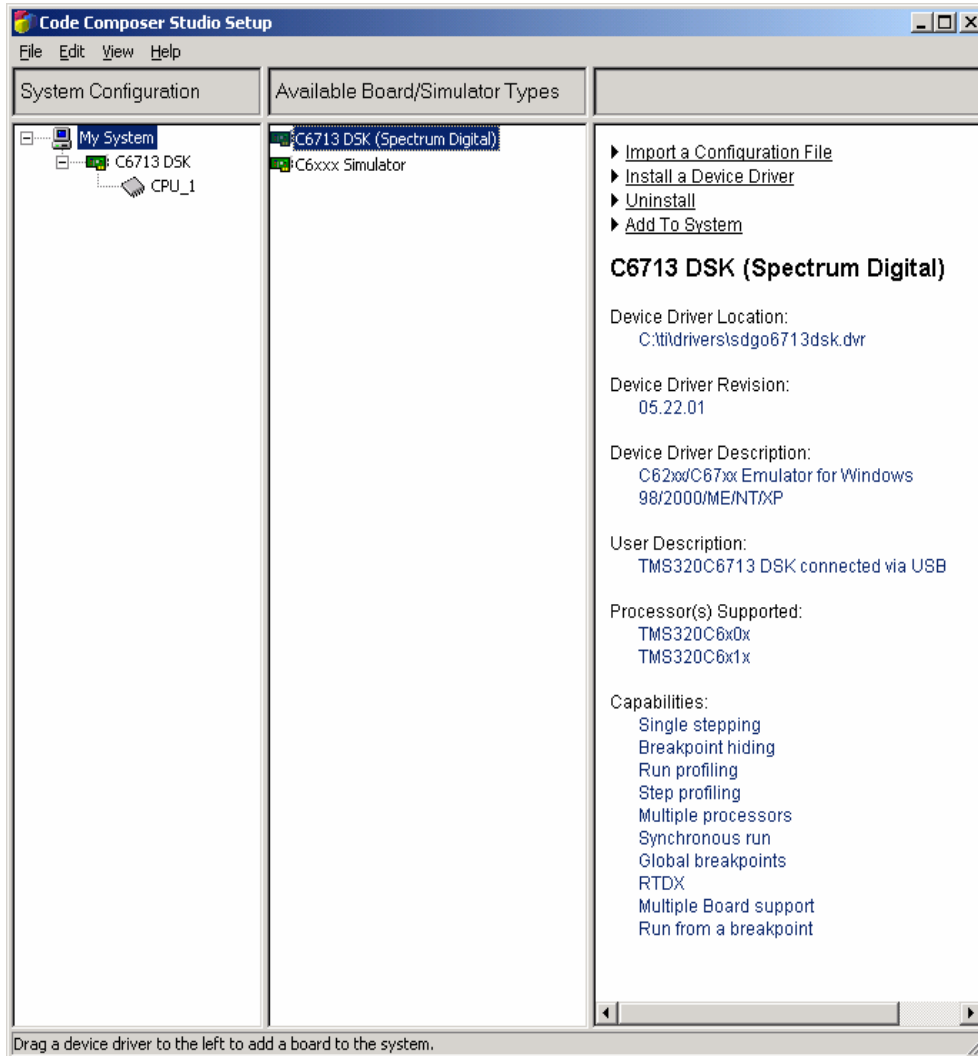


Figure 32. CCS Setup

There exists one command in Matlab called `ccsboardinfo`, that returns configuration information about each board and processor installed and recognised by CCS. `Ccsboardinfo` is type in the Matlab prompt to know if the board is correctly installed and recognised. Once typed this command the following output is obtained:

```
>> ccsboardinfo
```

Board	Board	Proc	Processor	Processor
<i>Num</i>	<i>Name</i>	<i>Num</i>	<i>Name</i>	<i>Type</i>
0	C6713 DSK	0	CPU_1	TMS320C6x1x

As can be seen, the board is correctly installed and recognised. We do not know why the board is not recognised by Embedded target but because of this a big problem arise; the model can not be targeted on the desired DSP board.

Before running the code as an executable, the run-time options for the model on the board must be configured. When the TI c6000 runtime from the select tree is chosen, the Build action option to specify to Real-Time Workshop what to do when Build is clicked can be seen. There are four possible build actions explained next:

- *Generate code only* directs Real-Time Workshop to generate C code only from the model. It does not use the TI software tools such as the compiler and linker, and you do not need to have CCS installed. This option does not apply to targeting with the Embedded Target for TI C6000 DSP. It just generates source code without building and executing the code on your target. It is not a very good option because of the use of the Generate code only option does not generate DSP/BIOS enabled code and thus not allowing us to use DSP/BIOS features.
- *Create CCS_Project* directs Real-Time Workshop to start CCS and populate a new project with the files from the build process. This option offers a convenient way to build projects in CCS. This option just builds a project, it does not try to target the model in the

board and run such model into the DSP board. Because of this, it does not probably really matter if the board is recognised or not.

- *Build* builds the executable file, but does not download the file to the target.
- *Build_and_execute* directs Real-Time Workshop to download and run the generated code as an executable on the target. This option can not be used because the Real-Time Workshop does not recognise the board, so it can not execute the generated code on the target.

After analysing all possible building actions we come to the conclusion that only *Create CCS_Project* and *Build* are suitable ones. *Build and execute* would not work for the reasons previously exposed and *Generate Code Only* is not valid for our purpose.

4.4.3 TESTS

4.4.3.1 FIRST TEST

Once loaded the System Target Configuration file *Ti_c6000.tlc*, the parameters are set as it is shown in the manual. Before starting with the code generation, some parameters must be set as the manual indicates. It is mandatory to set the fixed-step solver step size because Real-Time Workshop can not work with variable-step step size and to fix the simulation stop time to inf.

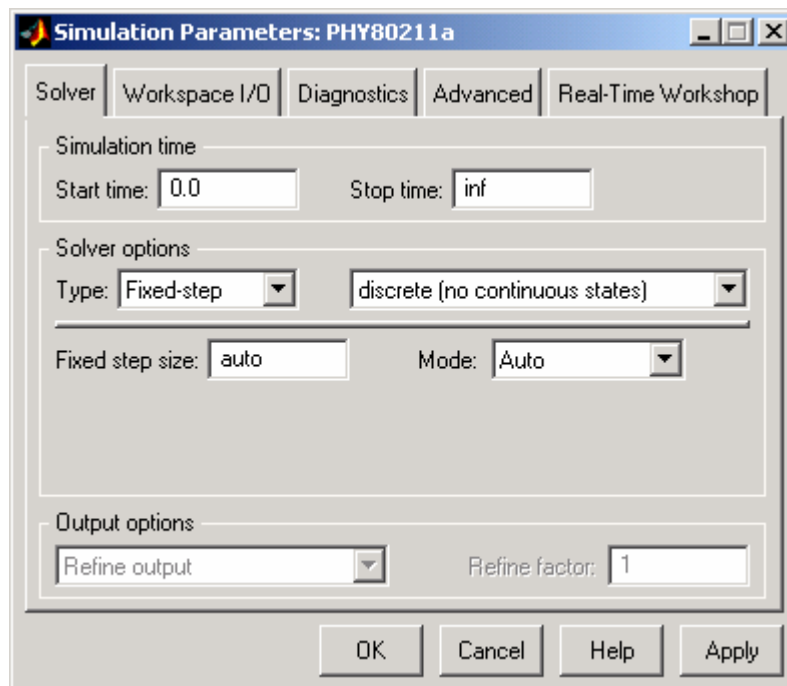


Figure 33. Simulation Parameters

The first test is focused in generating code for the transmission and receiver part. All the blocks and subsystems located in the transmission are selected and a new subsystem that includes all of them is created. Once the subsystem is created and the build option is set to build (same results are obtained by setting the build option to create CCS_Project), we are ready to get started with the code generation by clicking the build button. Once clicked, the next screen is shown:

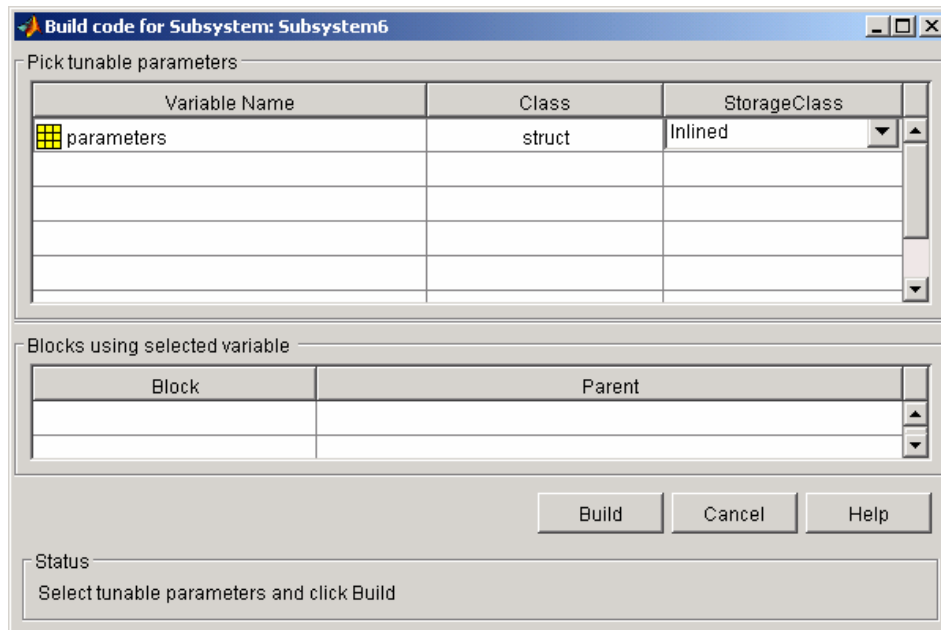


Figure 34. Parameters for the subsystem to build

Here you have to pick the parameters that are used for the subsystem. It is clear that the variable name `parameters` (defined parameters in `Wlan80211a.m`) must be chosen and click on **Build** one more time.

During the generation of the code the next error, shown in the figure below, comes up:

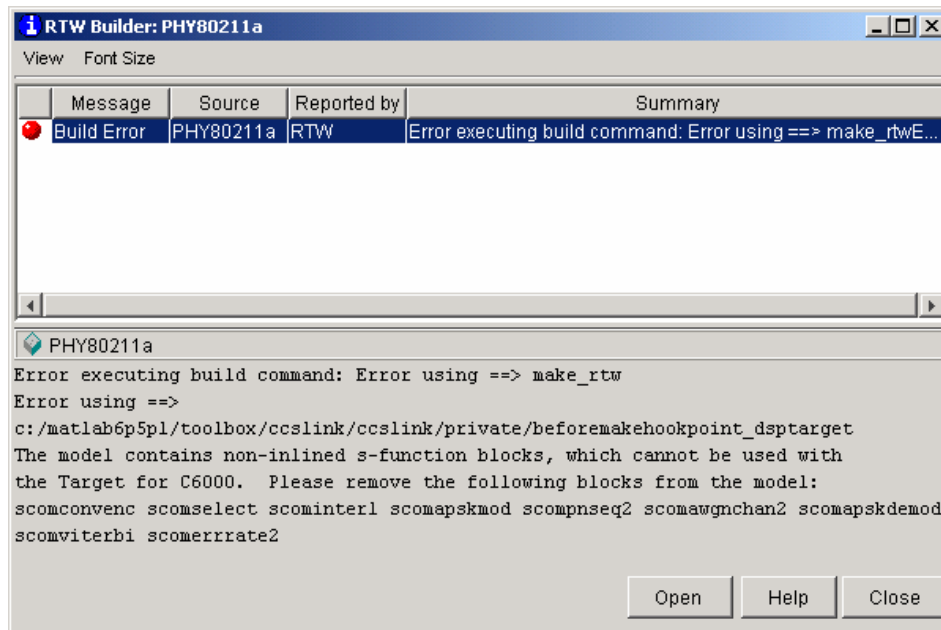


Figure 35. RTW builder error

According to this error message, blocks that contains those s-functions cannot be used with the target for c6000. We check which blocks contain these functions and the result of the search is that all of them are blocks used in the modulation subsystem, they belong to the Simulink's communication blockset library. In the manual is said that the model can be formed with blocks from the DSP blockset library as well as from the others libraries but this message contradicts such affirmation.

The reason that could explain the nature of this error message is that a trail and old version of the embedded target is being used. Perhaps this version lacks of some options or is incomplete, as it usually happens with trial versions, and some features/options are cut. Clearly, we are not able to generate code from the whole transmission or receiver model if blocks that contain non-inlined s-functions are removed.

On account of this, the next test is focused on the OFDM transmitter/receiver subsystem. It does not contain any block that uses non-inlined s-functions. The following test concentrates on generating code from the OFDM transmitter subsystem.

4.4.3.2 SECOND TEST

The OFDM transmitter subsystem is already built. This simplifies the task and the build option can be set to build again (same results are obtained setting the build option to create CCS_Project).

The program now does not complain about the use of non-inlined s-functions, mostly because this subsystem does not contain any of them, but another kind of error arise. The produced error is shown next:

```
### Generating the DSP/BIOS configuration file ...
js: "C:\ti\bin\utilities\tconf\tconfini.tcf", line 100: Couldn't open file
"C:\documentation".
Error using ==> make_rtw
Error using ==>
c:\matlab6p5p1\toolbox\ccslink\ccslink\private\generate_cdb_titarget
Error returned from TCONF.
```

The causes of this error are also unknown to us but after looking for some information on the internet some information can be compiled about it. One of the following causes could produce it:

These errors are usually due to a configuration problem. One possible solution is resetting the compiler path with the following instructions:

1. Type "xpcsetup" at the MATLAB prompt.
2. Set the compiler path correctly in the "compiler path" box.
3. Click "update".

Following this instructions, the mentioned command is typed in the Matlab prompt obtaining the next result:

```
>> xpcsetup
??? Error using ==> save
Unable to write file C:\MATLAB6p5p1\toolbox\rtw\targets\xpc\xpc\xpcenv.mat:
permission denied.

Error in ==> C:\MATLAB6p5p1\toolbox\rtw\targets\xpc\xpc\xpcinit.m
On line 32 ==>
save([xpcroot,'xpc\xpcenv'],'propname','actpropval','newpropval');
Error in ==> C:\MATLAB6p5p1\toolbox\rtw\targets\xpc\xpc\xpcsetup.m
On line 44 ==> xpcinit;
```

Another cause because this error could be produced is that this is a bug in the Embedded Target for TI C6000 DSP 1.0 (R13). This problem could be due to an incomplete installation of the Embedded Target for TI C6000 DSP. The software is correctly and completely installed, so can be concluded that there is nothing we can do for fixing this problem or that, at least, we were not able to find out how to do it due to a lack of information.

The building options that have been tried were *Build* and *create CCS_Project* but none worked. The only possible option left is the *generate code only* but, as mentioned before, this option is not the most appropriate for our purpose. Though, the next test is focused on generating code using the build option *generate code only* under the transmission subsystem.

4.4.3.3 THIRD TEST

As shown before, code of the transmission subsystem is going to be generated. All the blocks and subsystems located in the transmission are included in this subsystem. The *generate code only* option will be used this time. With this option, the process of code generation worked without giving any errors. Once having the source code files generated, the next step is to create a CCS project using these files. All the c, asm and cmd files must be added to the project. The header files are loaded automatically in the compilation process. One tip is to put all header files that are called from the c files in the same folder. For that, some header files from the Matlab folder must be located in the same folder where the other files are. If this is not achieved, some errors would appear under the compilation, advising us that the header files can not be opened. After the project has been created by selecting “create new project” from the project option, and adding all the necessary files (right clicking over the project name already created, and select add files to the project), we are ready to build the project. In the process of building the project a large number, up to 100, of errors appear. Under such circumstance we give up, it is not possible to fix 100 and more errors if you are dealing with a generated code and not with a code written by ourselves. After these negative results are obtained, one can ask himself some questions: Why are there so many errors? Is the software working properly? Are we doing something wrong?

After these failed attempts of generating useful code from the simulink model, all possible options of the Embedded Target software have been consumed and anyone of the build options we could allow us to generate any useful code. Because of these negative results, we decide to go in another direction.

- Generate the code with another system target file. Generic Real-Time Target(grt.tlc)

4.4.3.4 FOURTH TEST

Now another system target file (grt.tlc) is going to be used. This system target file generates c code from the simulink model as well but the code generated is not prepared for working under CCS.

The same way as before, the first attempt will be to generate code from the whole transmission subsystem. The subsystem was created before including all the blocks and subsystems of the receiver side. The next step is to set the simulator parameters, and as mentioned before, the chosen system target file is Generic Real-Time target (grt.tlc). The remaining parameters do not need to be changed, they are properly set. We are ready to build the code. By clicking the build bottom the code is generated. During the generation of the code no errors occur and the files are correctly built.

Once we have the code files created by Real-Time workshop, the task is to create a new project with CCS containing these together with another necessary files. After the source code files are generated, the next step is to create a CCS project using such files. All the c files created must be added to the project. The header files are loaded automatically in the compilation process. As previously said, one tip is to put all header files that are called from the c files in the same folder. For that, some header files from the Matlab folder must be located in the same folder where the other files are. If this is not achieved, some errors would appear under the compilation, advising that the header files can not be opened or accessed. Beside those files, a DSP/BIOS file (.cdb) must be created. Choosing the following option “New-> DSP/BIOS file” the next screen appears:



Figure 36. DSP/BIOS File

The option dsk6713.cdb is chosen. Once the .cdb file is modified, saved and added to the project, the configuration manager will create and load the following files:

- ♦ cfg.s62(Assembly file)
- ♦ cfg_c.c(C file)
- ♦ cfg.h(Header file for C)
- ♦ cfg.h62(Header file for assembly)

A file called cfg.cmd is also created but must be loaded by the user. This is used by the linker in order to allocate the different memory addresses to different sections of the application. After creating the project by selecting create new project from the project option, and adding all the necessary files (right clicking over the project name already created and select add files to the project), we are ready to build the project. During the process of building the project some errors appear while the code is compiled, errors produced for undefined variables and for some header files that could not be found during the compilation. However, the number of errors is not very large and these can be fixed.

After fixing the errors, our code is built again. During the compilation of the

code, and since the files are already corrected, no error comes up. The problem arises at the time of linking and loading the executable, as the following errors appears:

```
----- Transmission.pjt - Debug -----
[Linking...] "c:\ti\c6000\cgtools\bin\cl6x" -@"Debug.lkf"

"C:\DOCUME~1\luiza605\LOCALS~1\Temp\TI1036_4", ERROR! at
line 775:
[E0800]
    Offset too large
        STW    .D2T2  B4,*+B14(_rtDWork+54184) ; |48|

"C:\DOCUME~1\luiza605\LOCALS~1\Temp\TI1036_4", ERROR! at
line 780:
[E0800]
    Offset too large
        STW    .D2T2  B4,*+B14(_rtDWork+54188) ; |50|

"C:\DOCUME~1\luiza605\LOCALS~1\Temp\TI1036_4", ERROR! at
line 798:
[E0800]
    Offset too large
        STW    .D2T2  B4,*+B14(_rtDWork+54200) ; |61|

"C:\DOCUME~1\luiza605\LOCALS~1\Temp\TI1036_4", ERROR! at
line 1564:
[E0800]
    Offset too large
        LDW    .D2T2  *+B14(_rtDWork+54200),B6 ; |253|

"C:\DOCUME~1\luiza605\LOCALS~1\Temp\TI1036_4", ERROR! at
line 1602:
[E0800]
    Offset too large
        LDW    .D2T2  *+B14(_rtDWork+54200),B6 ; |260|
```

Continuing until the amount of 34 errors. We are unaware of the nature of this

error, but it seems to be because of a problem of memory allocation. Possibly, the cause could be a bad configuration of the .cdb file, but neither my supervisor nor other persons who I asked for help do not know anything about the nature of this error, not could anything about the possible causes of this error be found on the internet.

4.4.3.5 FIFTH TEST

After concluding that there is no possible way of fixing the previous error (or at least without having the knowledge to overcome it), the last test is focused in generating code from the OFDM transmitter subsystem. The same steps as in the previous test are followed for setting the parameters. The subsystem is built and the code is generated successfully, without the appearance of any errors during the process. But this was already expected, the hard part, to make it work in the CCS, comes next.

As done several times during the former tests, a new project is created. All suitable files are added, this includes the c files, the cdb and the cmd. The project with all necessary files is created, now we are ready to get started with the building of the project. As usual, some compilation errors appear during the building process. Like in the previous test, most errors are because of undefined variables or headers files that can not be loaded. The rest are also other easy to correct errors. The errors are corrected in the best possible way. The compilation errors are already corrected but during the linking process a new error appears and the executable file can not be built. The error leads to the following message:

```
----- OFDM_Trans.pjt - Debug -----
[Linking...] "c:\ti\c6000\cgtools\bin\cl6x" -@"Debug.lkf"

undefined                first referenced
symbol                   in file
-----
_MWDSP_PadAlongCols
C:\ti\myprojects\OFDM_Trans\Debug\Subsystem1.obj
_MWDSP_R2DIT_TBLS_Z
C:\ti\myprojects\OFDM_Trans\Debug\Subsystem1.obj
_MWDSP_ScaleData_DZ
C:\ti\myprojects\OFDM_Trans\Debug\Subsystem1.obj
```



```

_MWDSP_R2BR_Z
C:\ti\myprojects\OFDM_Trans\Debug\Subsystem1.obj
_main               c:/ti/c6000/bios/lib/biosi.a62
>> error: symbol referencing errors - './Debug/OFDM_Trans.out' not built

```

*Build Complete,
1 Errors, 0 Warnings, 0 Remarks.*

The error message says that those symbols are undefined. Those symbols are searched in the files starting with the first one, `_MWDSP_PadAlongCols`, and is found that this method is defined in the header file `dsppad_rt.h`, header included in `Subsystem1_private.h` (`Subsystem1` is the name of the OFDM transmitter subsystem, all the generated files from that subsystem by Real-Time Workshop get that kind of names, `Subsystem1.c`, `Subsystem1_data.c`,...), header file that in his turn is included in `Subsystem1.c`. The meaning of this is that `_MWDSP_PadAlongCols` is already defined in the code, otherwise a compilation error should be arised. But examining the map file created in the compilation, the next message can be seen:

```

UNDEFED      _MWDSP_PadAlongCols
UNDEFED      _main
ffffffff     _CACHE_L2
UNDEFED      _MWDSP_R2BR_Z
ffffffff     _CACHE_L2
UNDEFED      _MWDSP_ScaleData_DZ
UNDEFED      _MWDSP_R2DIT_TBLS_Z
00010c88     TSK_idle
00010c88     _TSK_idle
00010ce0     TSK_idle$stack
00011078     TSK_idle$stkptr
000110e0     LOG

```

No memory address is kept for those symbols, being the cause unknown, but is clear that somehow they are not defined because no memory is allocated to them. As it is a generated code, it is quite hard to understand what is the use of these functions and how they could be modified or defined in a correct way to solve those errors. Therefore, this problem can not be fixed.

4.4.4 CONCLUSIONS AFTER THE ACHIEVED TESTS

After the failed achieved attempts described previously, some conclusions are reached. It is clear that the software, Embedded target, is not working properly. This could be due to the next reasons:

- The board is not configured as it should be or is not supported. When the system file target is selected a warning message is shown, advising us that no desired board could be found. Perhaps the board is not supported because a trial and old version of the Embedded Target for c6000 is being used.
- The software is a trial and old version, some features could be limited only to the complete version, but this is not referred to in the manual.
- Possible failures during the procedure. Something, like the Simulink model, could be incorrectly made. Another option could be a bad configuration of the options of the system target file.

After all this failure tests one question could come up to everyone reading this report: why did we choose Embedded Target for generating code from the model instead of building the code ourselves? The answer is that the goal of the project was to do a system performance analysis of the model running in both platforms Simulink and CCS (in the CPU or in a DSP processor). Once we had the Simulink model built we needed to implement that in CCS and Embedded Target was the faster way for doing that. The problem is that it did not work as it was expected, probably because of one of the reasons explained above.

For future work, an advice is to try to get the current version of the software and install everything correctly, ensuring that everything is working properly before starting. It would be interesting to continue with this simulink model and maybe improve it a bit. It should work trying to target the OFDM transmitter and receiver subsystem first on the DSP board with the new version of the software. If it is still not working, the only way is implement the own code. For this purpose there are some useful libraries, that can be very useful to fulfill this assignment, that everybody can access on the TI webpage. It is hard to do but possibly this is the best and only way if the Embedded Target is still not working because when problems and errors arise with the generated code this is harder to fix than if these errors arise with an own written code. It is easier to

debug because we already know how the code is built and where the errors come from.

5. FFT IMPLEMENTATION & PERFORMANCE ANALYSIS

5.1 FFT IMPLEMENTATION

As explained in the previous chapter, targeting the Simulink model on the Dsp board was an impossible task to accomplish. In order to get some results, a basic block in a wireless communication system is implemented for running in the DSP processor, the FFT.

The Fast Fourier Transform (FFT) is an efficient computation method of the Discrete Fourier Transform (DFT) and one of the most important tools used in digital signal processing applications. Because of its well-structured form, the FFT is a benchmark in assessing digital signal processor (DSP) performance. The development of FFT algorithms has assumed an input sequence consisting of complex numbers. This is because complex phase factors, or twiddle factors, result in complex variables. Thus, FFT algorithms are designed to perform complex multiplications and additions.[6]

The goal is to implement the FFT algorithm in CCS, and, once implemented, run it and make a system performance analysis between the implementation in Matlab and the implementation in CCS. In other words, the aim is to compare the implementation running both in the CPU and in a DSP processor.

For achieving this task, we are going to take advantage of the facilities of the DSPlib (dsp library) and specifically of the function *void DSPF_sp_cfft2_dit(float * x, float * w, short n)*, using it for creating the function *fft(float * x, float * w, short n)*.

This routine performs the decimation-in-time (DIT) radix-2 FFT of the input array "x". "x" has N complex floating-point numbers arranged as successive real and imaginary number pairs. Input array "x" contains N complex points (N*2 elements). The number of complex points, N, will be 64, since this is the number of points of the FFT in the model. Therefore, the size of the input x, will be 128, 64 real and imaginary number pairs. The input values are generated by the next equation, Matlab is used to perform this task (The values of X can be seen in appendix B)[6]:

$$x = \cos(0:\pi/10:\pi*655/2-1);$$

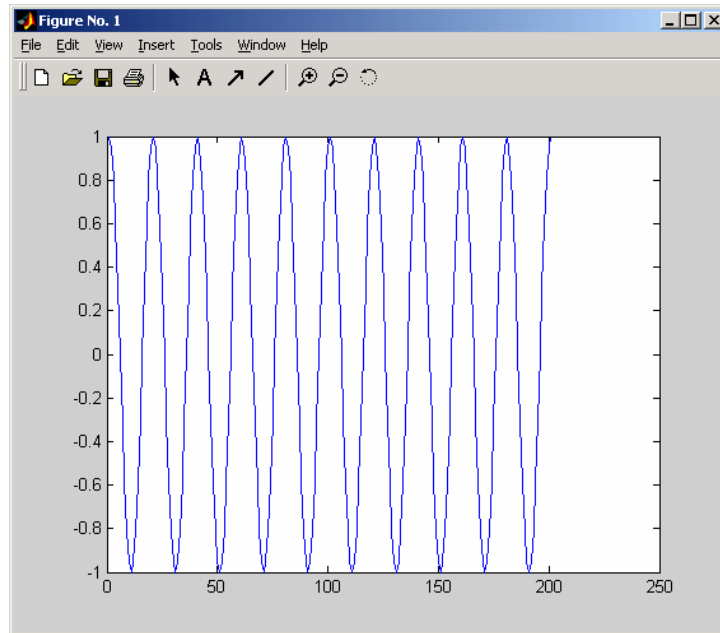


Figure 37. Signal generated by the equation

The coefficients for the FFT are passed to the function in array “w” which contains N/2 complex numbers (N elements) as successive real and imaginary number pairs. The FFT coefficients “w” are in N/2 bit-reversed order. The elements of input array “x” are in normal order. The coefficients are generated as follow:

```
n = 64;
double delta = 2 * PI / n;

for(i = 0; i < n/2; i++)
{
    w[2 * i + 1] = sin(i * delta);
    w[2 * i] = cos(i * delta);
}
```

After creating the array, “w” must be bit-reversed using bit_rev.

```
void bit_rev(float *x, int n);
```

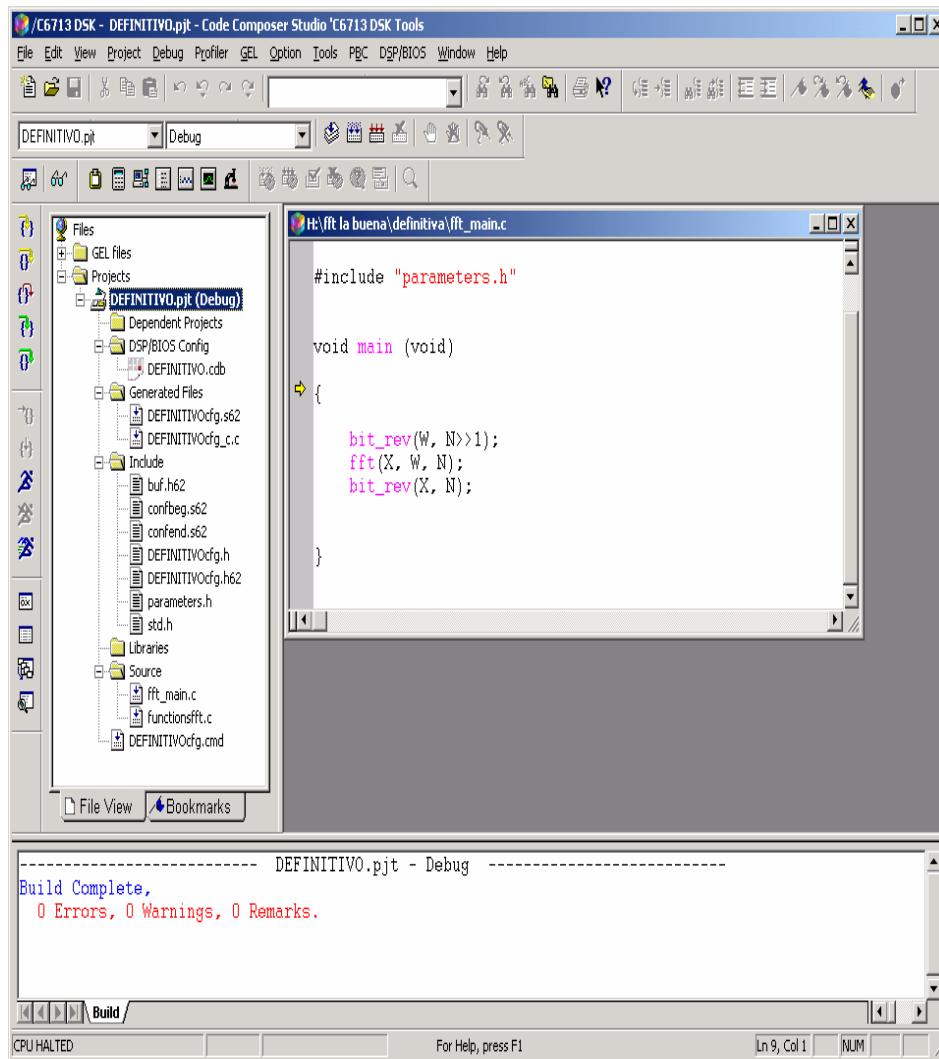
This routine bit reverses the floating point array “x” which is considered to be an array of complex numbers with the even numbered elements being the real parts of the complex numbers while the odd numbered elements being the imaginary parts of the complex numbers.

After running the function `fft( x, w, n)`, the output will be also stored in “x”. The output must be bit-reversed as well, using the bit reverse function: `bit_rev`.

Once having implemented the code for the FFT algorithm we are ready to create a project in CCS. Once a new project is created, the two c files are added:

- `fftfunctions.c` (file that contains the functions `fft` and `bit_rev`)
- `fft_main.c` (main)

As explained before, the header file `parameters.h`, is loaded directly. A DSP/BIOS file must be created as well and must be added to the project together with the `cmd` file. In the next figure, an overview of the project is shown (DEFINITIVO.pjt). Once all the necessary files were added to the project, it is time to build it. During the build process no error arise and, after being built, the executable file is loaded because the option load after building was activated. The FFT algorithm works properly, we are ready to get started with the analysis of the system performance.

**Figure 38.Overview of the project**

5.2 ANALYSIS

With the FFT implementation done in both platforms, we are ready to start comparing the running performance of both. For this analysis the profiler is going to be used both in Matlab and in CCS.

The analysis is started in the Simulink model. For such analysis the profile function is going to be used, as mentioned before. For starting it, the option profiler must be chosen in the view menu. Once this function is chosen the profiler window will open. In the profiler window there is a command prompt where you can type the code to run. The task is to run the model, so the command `sim PHY80211a` is typed. After this, the Simulink model is loaded and run. Once the execution of the model is stopped, the profile summary is obtained, as shown in the next figure. The filename `dspblkfft2` is emphasised because this is the m-function used by the fft block. As it can be seen, this is the second function in terms of execution time ($0.070/2 = 0.035$).

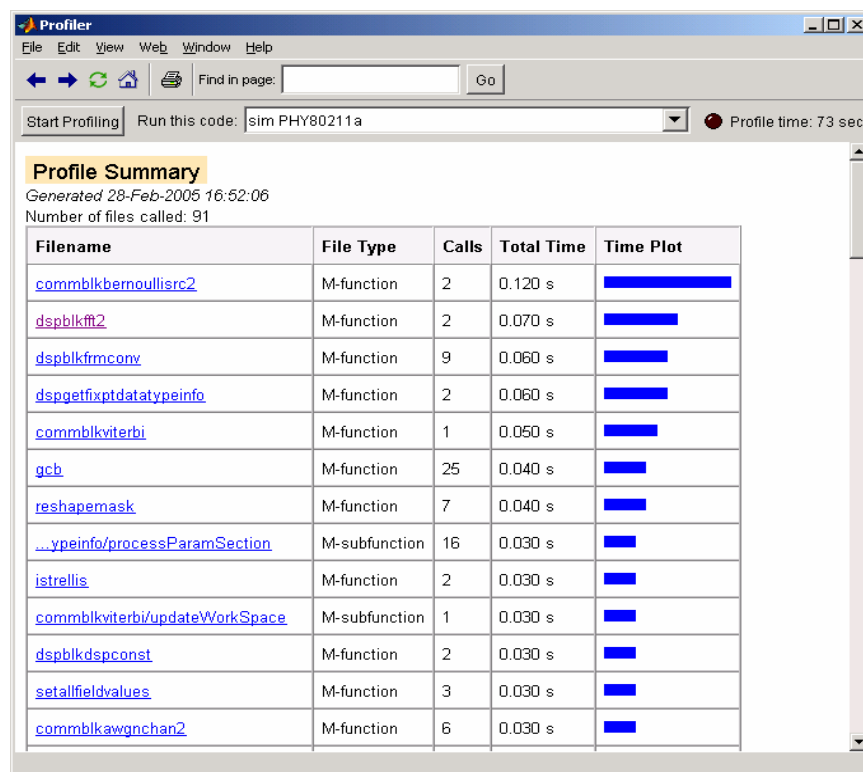


Figure 39. Matlab profiler

Typing in the Matlab prompt profile report, a report of the performance is obtained in a html file. This is shown next:

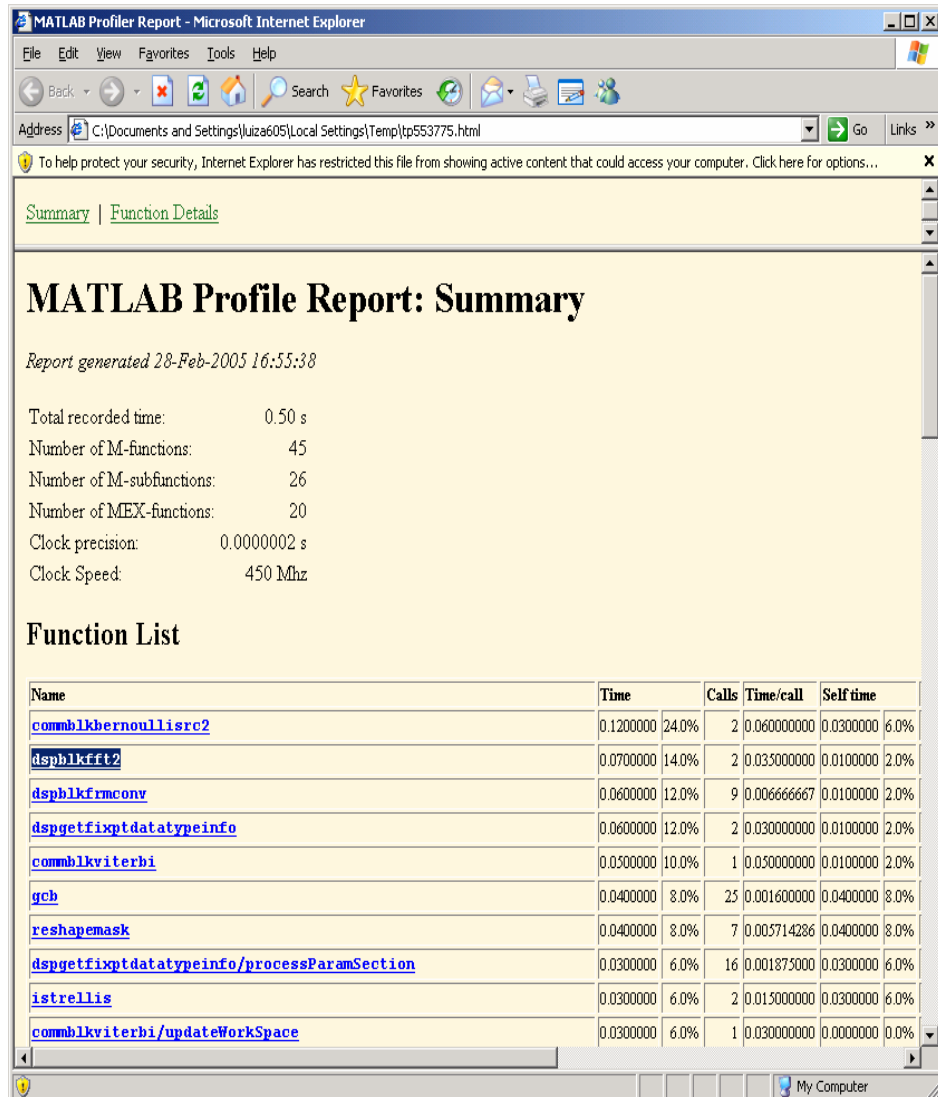


Figure 40. Profile report

The total time for executing the model is 0.5 seconds. 14 % of this time is used in executing the FFT functions (FFT/IFFT), so, as explained before, the time for the FFT block is 0,035 seconds. A more detailed analysis of the functions can

be seen by clicking over the function name. The function details are shown next.

Recall that the clock speed is 450 MHz and the clock speed of the DSP board is 225 MHz. It should be taken into account in order to compare both performances.

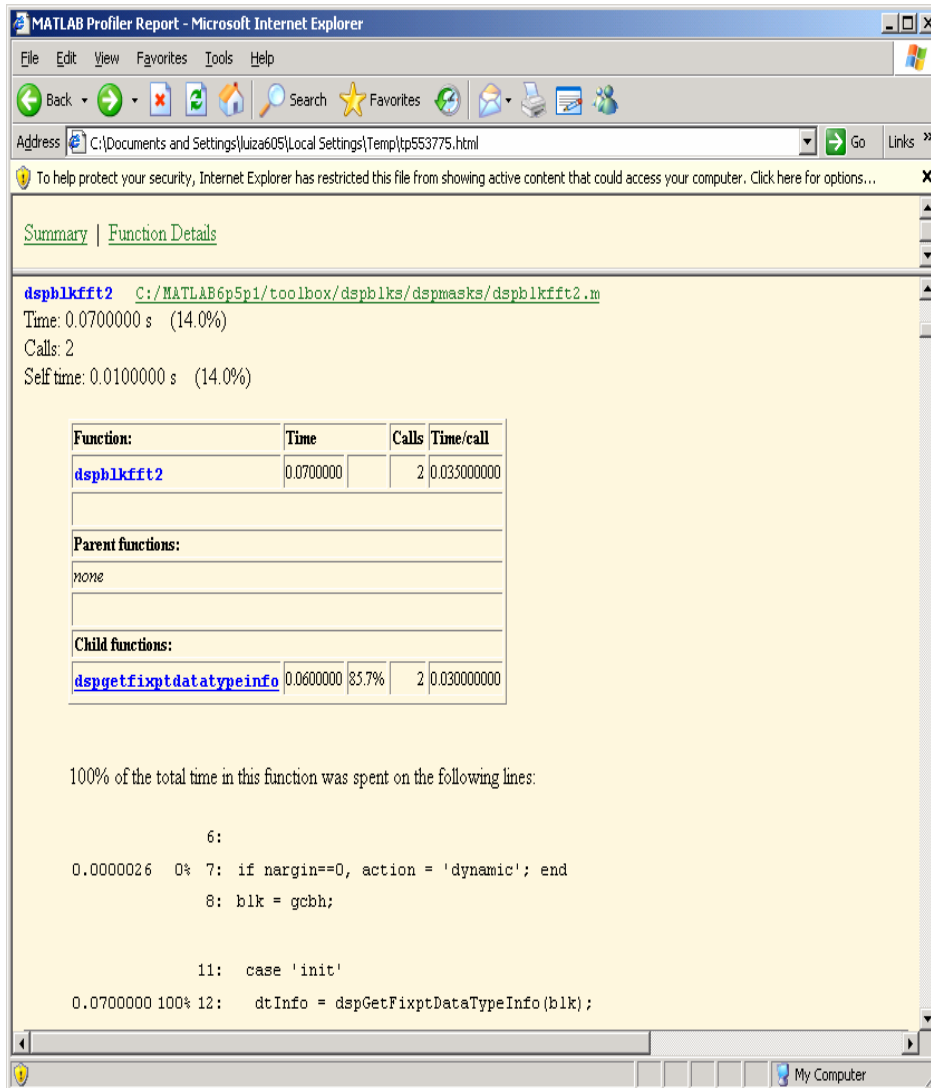


Figure 41. Dspblkfft2 function details

After looking carefully to the standard it can be seen that the T_{FFT} (IFFT/FFT period) is 3.2 microseconds (0.0000032 seconds). The value obtained for T_{FFT} in Simulink was quite far away from the time requirements shown in the standard.

Now is time to analyse the FFT running on the DSP processor. For this purpose the profiler tool available in CCS is used. After the project has been build, it is time to load the profiler starting a new session. This is achieved choosing the option start new session from the profiler menu. A new session is started and, after this, the functions have to be loaded to the profiler and set the profiler control properties to see the results in nanoseconds.

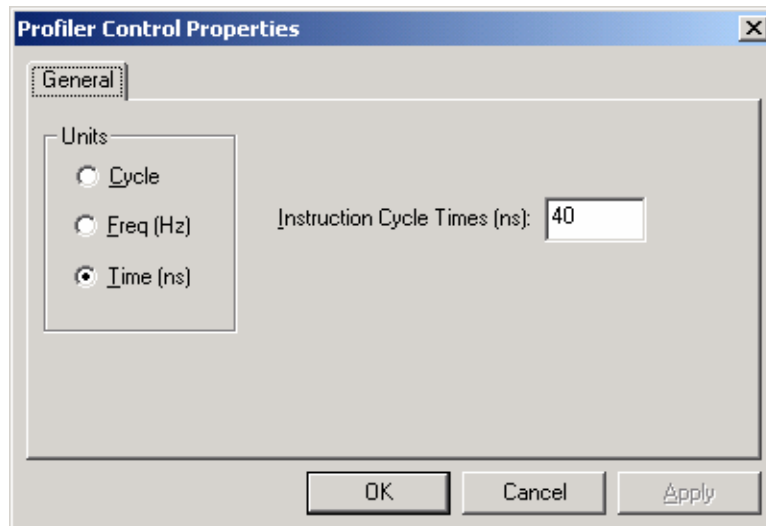


Figure 42. Profiler properties

The next step is to set two breakpoints, one at the beginning of the main function and the other at the end as depicted next (the red points are the breakpoints) to control the execution of the project:

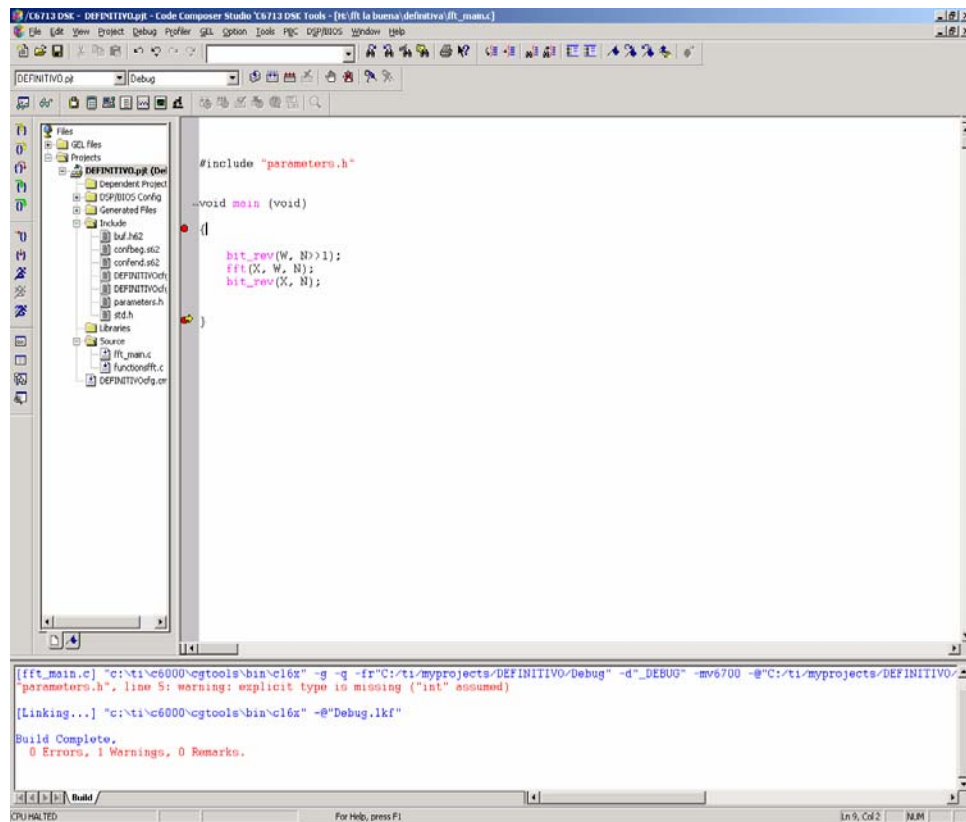


Figure 43. Breakpoints

After adjusting the necessary settings for obtaining correct results from the profile, the project is run and the next results are obtained with the profiler.

The results can be clearly seen in the next table. The executed code is the next one:

FUNCTION	CODE SIZE	COUNT	INCL.TOTAL	EXL.TOTAL
Main()	116	1	0.001928400	0.000000840
Bit_rev()	488	2	0.000458280	0.000458280
Fft()	848	1	0.001468680	0.001468680

Table 3. Profiler results

The time in the table is shown in seconds instead of in nanoseconds as shown by the profiler. They give information about the code size of the function, the number of times that is called during the execution, the total time for running the whole function (**Incl. Total**. Main, for instance, counts the time for executing main and the functions that it includes, bit_rev and fft) and the own time for running the function (**Exl.total**. It counts only the main function and not the functions that are called inside main).

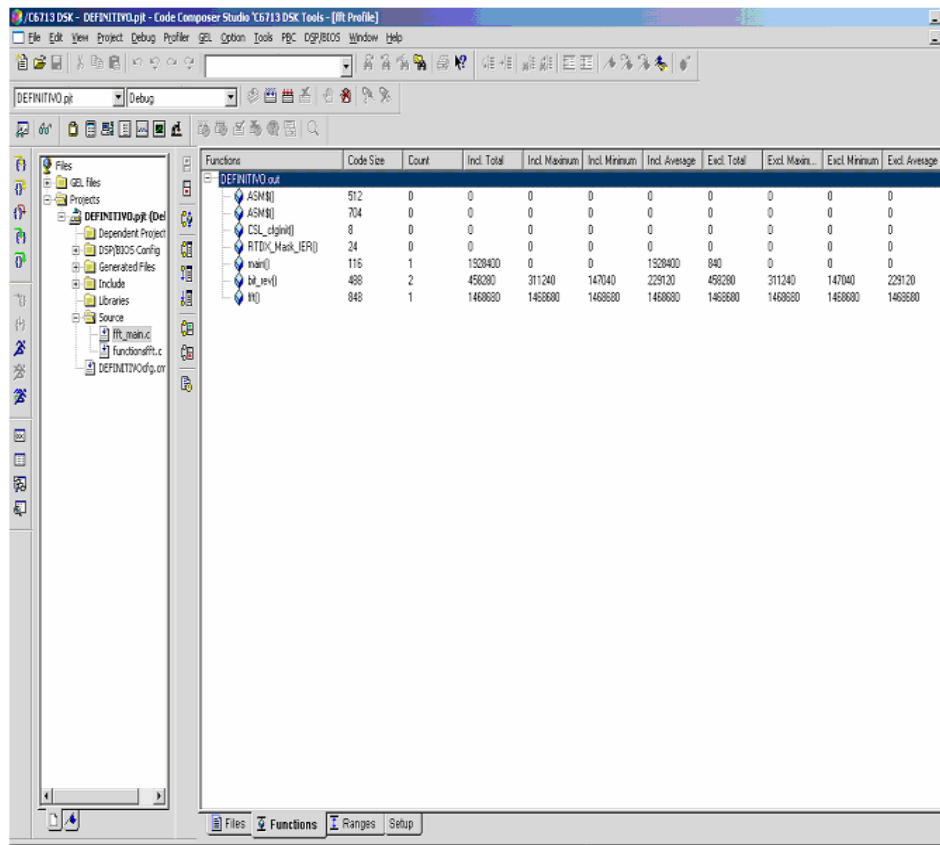


Figure 44. Profiler results

Analysing the results obtained, can be seen that the code size is of 1452 words, and that main and fft functions and bit_rev are called once during the execution. The time needed by the DSP for executing the FFT algorithm is, as can be seen in the table, of approximately 0.002 seconds. In comparison with the time obtained by running the FFT in Simulink that was 0.035 seconds, the difference is remarkable.

Analysing these results can be clearly seen that the DSP processor is much faster. In fact it is 17,5 times faster executing the FFT. It is also good to remark that the CPUs clock speed is twice that of the DSP.

Another way for estimating the execution time of the FFT algorithm can be, instead of starting a new profiler session, to enable the clock. In the profiler options choose enable clock, and, afterwards, view clock must be chosen before running the project. Set up the clock for setting the count type as cycles. One cycle takes 40 ns.

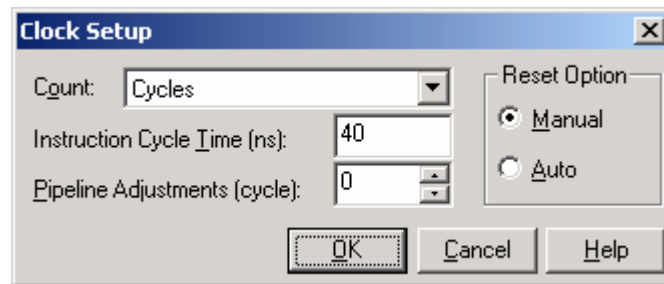


Figure 45. Clock Setup

We set to breakpoints as explained before and afterwards run the project. After running, the value of the clock is the one shown in the next figure.

There are slightly differences with the one got before, but, in general, is almost exactly the same. The value obtained is 48218 ns that converted to seconds is 0.001928720 s. Before a value of 0.001928400 was obtained; the difference is minimal.

Based on the results obtained can be said that the DSP processors are much faster than a normal microprocessor, at least at the implementation, where mathematical operations are involved. As explained in the DSP introduction, this is quite normal due to the features of the DSP processors. DSP processors arithmetic speed is very high compared with microprocessors. DSP procesors

usually have hardware adders and multipliers which can be used in parallel within a single instruction. This way both, an addition and a multiplication, can be executed in a single cycle.

The value obtained with the DSP processor, can be seen clearly, that is better than the obtained with a normal microprocessor, but this value is not that good if is compared with the Timing related parameters of the standard. A real time application could not be done because as it shown in the standard the IFFT/FFT period time is 3.2 microseconds (T_{FFT}) and the result obtained with the DSP processor is 0.001928400 seconds. This time could be improved, there are so many tools available in Code Composer Studio for debugging and for improving the performance of the code. Assembly code can be used instead of implementing the algorithm in c language as well, achieving those things a better performance of the FFT algorithm would be obtained.

Though have to be taken into account that the microprocessor used is not the faster one, like the graph next shows. Compared with other microprocessors it can be seen that this is rather slow.

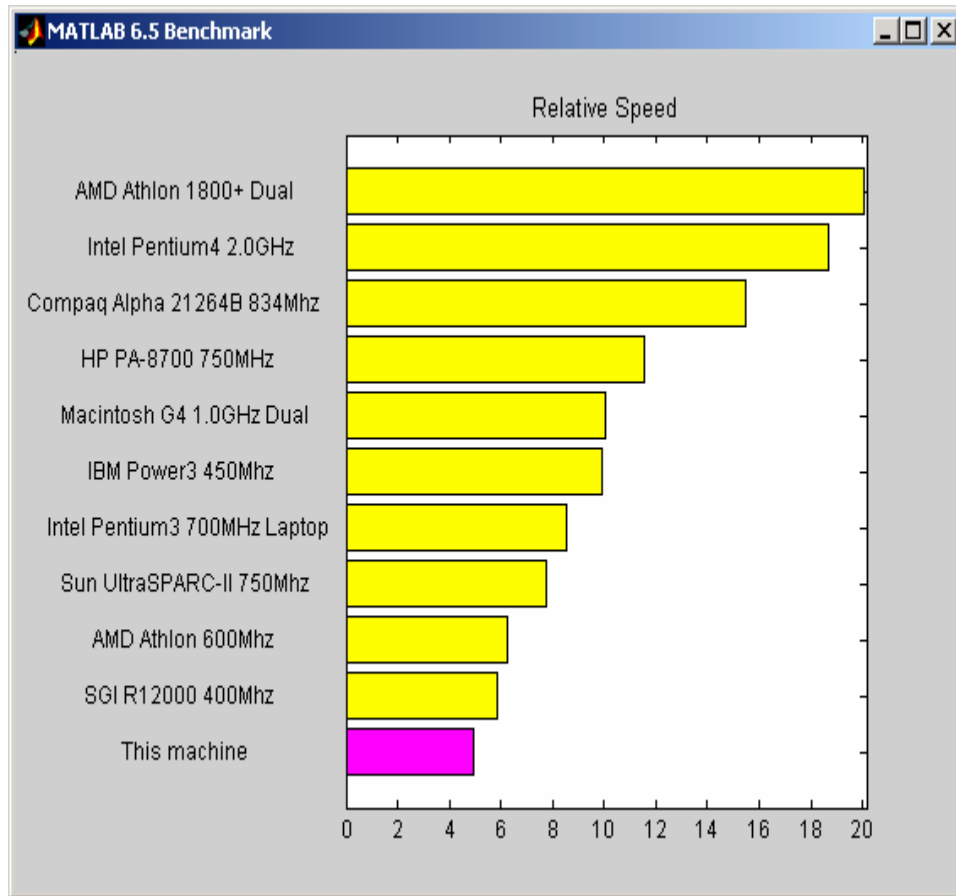


Figure 46. Matlab Benchmark

The task manager is used to see the performance of terms of memory. In the next figures the memory usage before running the model and while running the model under Simulink can be seen. Before running the model, the memory usage by matlab is 4796 K and while running the model this value increases till 28984 K. So, the memory usage for running the model is 24188K. The problem is here that it is pretty hard to estimate how much memory is dedicated to compute the fft block. For that, the memory measure for the FFT block can not be done exactly.

The same task is performed while running the FFT algorithm in the DSP processor and it can be seen that this task does not use any memory. This is due to the fact that the DSP board has its own internal memory. Because of this almost no CPU memory is used to perform the task.

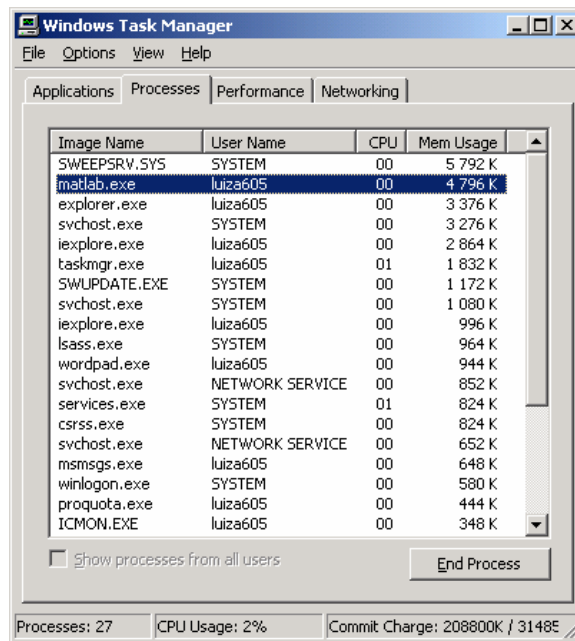


Figure 47. Before running the Simulink model

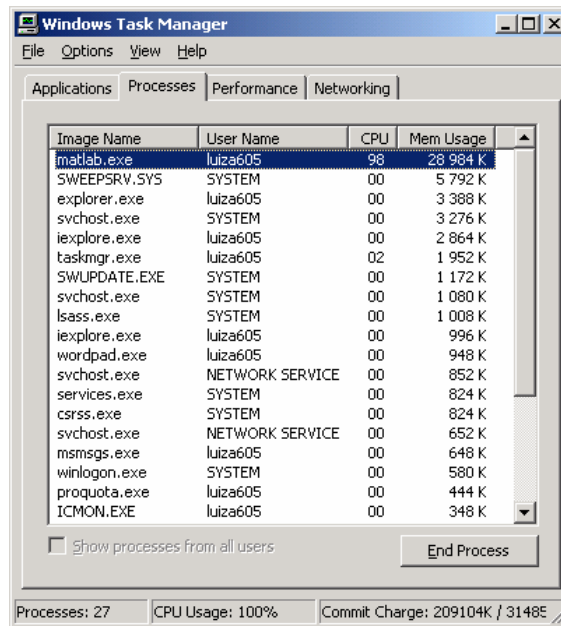


Figure 48. While running the Simulink model

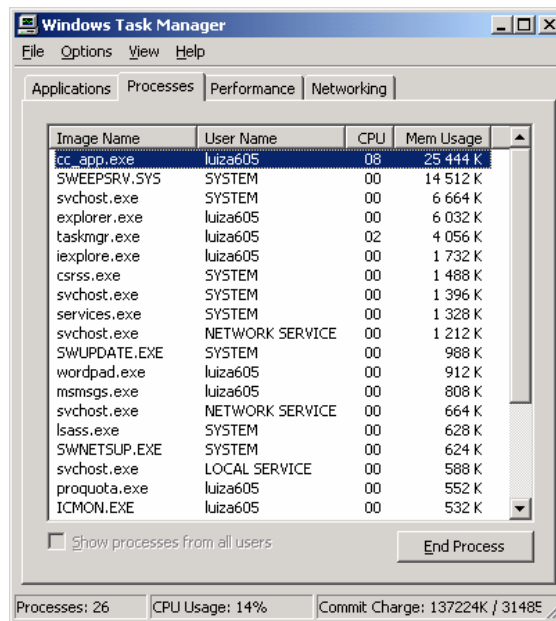


Figure 49. Before running the FFT algorithm

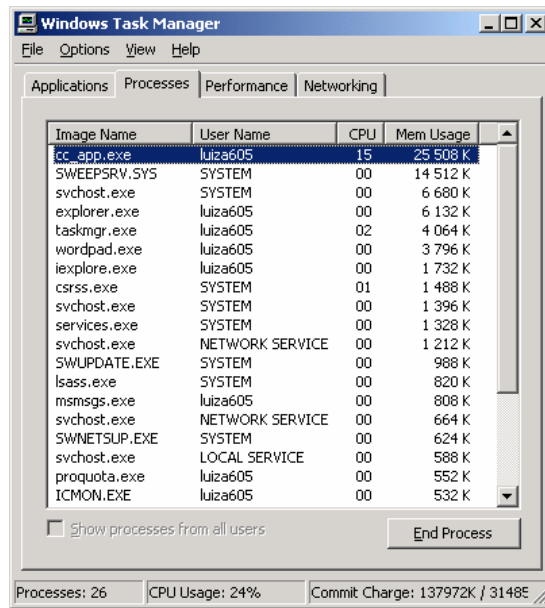


Figure 50. While running the FFT algorithm

In terms of CPU usage or CPU load, can be seen that the use of the CPU by the simulink implementation is much larger compared to the use of the DSP. This can be clearly seen in the next figures:

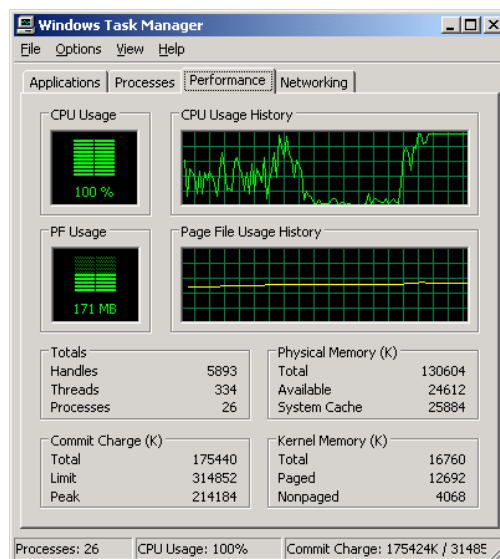


Figure 51. CPU usage of simulink model

In the next figure the CPU load while running the FFT algorithm in the DSP processor can be seen. The rate is very low.

The CPU is doing some work when any of the following events are occurring:

- hardware interrupts are serviced
- software interrupts and periodic functions are run
- task functions are run
- user functions are executed from the idle loop
- HST channels are transferring data to the host
- real-time analysis data is uploaded to the DSP/BIOS Analysis Tools

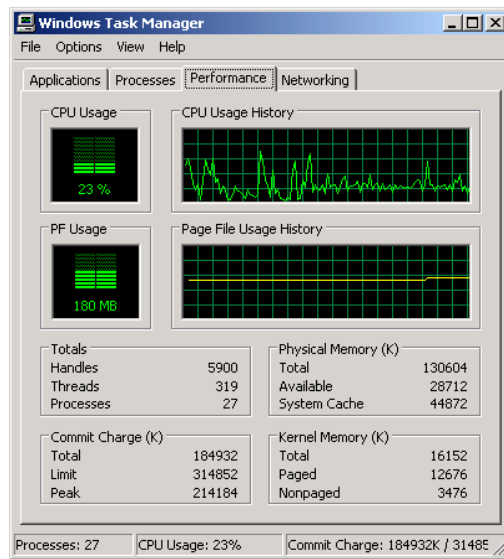


Figure 52. CPU Load Graph

6. FINAL CONCLUSION AND FUTURE WORK

One of the purposes of the thesis, to target the whole simulink model on the dsp board, could not be achieved. The cause of this is not clear; it could be due to the fact that the software used was not the suitable one. Another option could be that there was a wrong use of such software. A third possibility is that there was a problem with the configuration.

For future works I encourage people to continue trying with Embedded Target, using the current version, or, at least, not with a trial version but with the complete one. One hint for future works is to try to implement an OFDM transmitter and receiver and target this on the DSP using the software Embedded Target for c6000 but ensuring, before doing this, that everything is already installed and working properly, including the board, CCS, Matlab and Simulink. And if the Embedded Target is still not working, one advice is to generate your own code. Some libraries can be downloaded from the TI page (see references at the end of this work) with very useful functions that makes code generation much easier.

The whole model in the dsp could not be run but one important block in a wireless communication system, the FFT, was implemented and run in the DSP processor and in the CPU. By achieving this the following interesting conclusions are reached. As it was commented in the introduction about digital signal processors, that kind of processors have some advantages compared to normal microprocessors in terms of speed or memory usage. Due to the features of the DSP processor, they are quite effective in implementations where mathematical operations are involved, for example in a wireless system. As we could see in the results obtained in the analysis of the system performance, the DSP processors are much faster than a normal microprocessor and require less memory usage. For this reason the use of DSP processors is highly recommended for this kind of implementations. The DSP offers a large number of possibilities for improving the performance of the implementation as well, like debugging, memory allocation or synchronization techniques. We can conclude that if in the FFT implementation the best results are obtained by using DSP the same can be expected in a whole transmission wireless system. In any case, it would be interesting to see the results to confirm this theory.

7. ABBREVIATIONS AND ACRONYMS

BPSK	Binary phase shift keying
FFT	Fast Fourier Transform
GI	Guard interval
IFFT	Inverse Fast Fourier Transform
OFDM	Orthogonal frequency division multiplexing
PER	Packet error rate
BER	Bit error rate
QAM	Quadrature amplitude modulation
QPSK	Quadrature phase shift keying
ADSL	Asymmetrical Digital Subscriber Loop
DSSS	Direct Sequence Spread Spectrum
FHSS	Frequency Hopping Spread Spectrum
LAN	Local Area Network
MAC	Medium Access Control Layer
N_{bpsc}	Number of coded bits per subcarrier
N_{cbps}	Number of coded bits per OFDM symbol
N_{dbps}	Number of data bits per OFDM symbol
PHY	Physical Layer
PLCP	Physical Layer Convergence Procedure

PLME	Physical Layer Management Entity
PMD	Physical Medium Dependent
PPDU	PLCP Protocol Data Unit
PSDU	PHY Sublayer Service Data Units
QAM	Quadrature Amplitude Modulation
QPSK	Quadrature Phase Shift Keying
SAP	Service Access Point
WLAN	Wireless Local Area Network
CCS	Code Composer Studio

8. REFERENCES

- [1] TMS320C6000: *Code Composer Studio Tutorial*. Texas Instrument. February 2000.
- [2] Juha Heiskala and John Terry. *OFDM Wireless LANs: A Theoretical and Practical Guide*. SAMS, 2002.
- [3] *Embedded Target for the TI TMS320C6000 DSP Platform (For use with Real-Time Workshop (Version 1))*. July 2002.
- [4] Kent Palmkvist. *TSTE02 System Design CDIO (Lecture's Slides)*. Linköping University (Department of Electrical Engineering).
- [5] Steve Blonstein and Maher Katorgi. *ExpressDSP for Dummies*. Texas Instrument. 2003.
- [6] *TMS320C67x DSP Library Programmer's Reference Guide*. Texas Instrument. February 2003.
- [7] *SIMULINK-Dynamic System Simulation for Matlab (Using Simulink Version 3)*. The Math Works Inc.
- [8] *Code Composer Studio User's Guide*. Texas instrument. February 2000.
- [9] Carlos Alonso Guillen. *Implementation of the IEEE 802.11a MAC Layer in C Language*. Bachelor Thesis. Linköping University 2004.
- [10] Tom Karygiannis and Les Owens. *Wireless Network Security: 802.11, Bluetooth and Handheld Devices*. National Institute of Standards and Technology. November 2002.
- [11] www.ti.com
- [12] *OFDM Basics for Wireless Communications*. Darmstadt University of Technology.
- [13] Hector Uhalte Bilbao. *Dab Transmission System Simulation*. Master Thesis. Linköping University 2004
- [14] Chia-Sheng Peng. *WLAN 802.11a OFDM Transceiver*. TWT. 2001/11/22

- [15] Plamen Nedeltchev, PhD. *Wireless Local Area Networks and the 802.11 Standard*. March 31, 2001
- [16] Raúl Mateos. *Tutorial de Code Composer Studio*.
- [17] Özlem Kalinli. *Signal Processing with DSPs*. Undergraduation Thesis. June-2001
- [18] Tuukka Toivonen. *Digital Signal Processing Laboratory Work 521485S Laboratory Exercises with TMS320C6711 DSK*. Department of Electrical and Information Engineering. Information Processing Laboratory. University of Oulu. November 30, 2004.
- [19] Denis Real. *Graphical programming of DSPs*. Master's Degree Project Stockholm, Sweden.
- [20] *Signal Processing Examples Using the TMS320C67x DSPLIB*. Texas Instrument. August 2003.
- [21] *TMS320C6416/C6713 DSK One-Day Workshop (Student Guide)*. Texas Instrument.
- [22] *IEEE Standard 802.11a*. IEEE Standards Board. 1999 Edition. Piscatawa, USA.
- [23] *IEEE Standard 802.11a White Paper*. IEEE Standards Board.
- [24] Javier Atencia, Raul Nestar. *Aprenda Matlab 6.0 como si estuviera en primero*. Escuela Superior de Ingenieros Industriales.

9. APPENDIX

9.1 APPENDIX 1

Wlan80211a.m

```
function parameters = Wlan80211a(...
    SymbolPerFrame, ...
    SignaltoNoise, ...
    Inputpwr);
```

```
parameters.Symbols_per_frame = SymbolPerFrame;
%Number of OFDM symbols per packet or frame
parameters.OFDMTrainingSeqPerFrame = 2;
%The preamble is two long training sequences
parameters.OFDMTotalSymPerFrame = parameters.Symbols_per_frame
+ ...
    parameters.OFDMTrainingSeqPerFrame;%Number of symbols per
frame

parameters.Ndbps = 216; %Data bits per OFDM symbol
parameters.Nbpsc = 6; %Coded bits per subcarrier
parameters.Nsd = 48; %Number of data subcarriers
parameters.Nsp = 4; %Number of pilot subcarriers
parameters.Nfft = 64; %Number of FFT subcarriers
parameters.Ncbps = 288; %Coded bits per OFDM symbol
parameters.Ncyclic = 16; %Number of subcarriers of the cyclic prefix
parameters.NTotal = parameters.Ncyclic + parameters.Nfft;
%Number of subcarriers after the cyclic prefix addition
parameters.LongTrainingSequence = ...
    [1 1 -1 -1 1 1 -1 1 -1 1 1 1 1 1 -1 -1 1 1 -1 1 -1 1 1 1 0 ...
    1 -1 -1 1 1 -1 1 -1 1 -1 -1 -1 -1 -1 1 1 -1 -1 1 -1 1 -1 1 1 1].';
%Long training sequence used in the preamble, two sequences form the
%preamble
```

```
parameters.OFDMPreable =  
repmat(parameters.LongTrainingSequence, 1, ...  
    parameters.OFDMTrainingSeqPerFrame );  
%Preamble added in the beginning of the frame  
parameters.OFDMSymbolDuration = 4e-006; %Duration of one OFDM  
symbol  
parameters.PunctureVector = [1 1 0 1 1 0].';  
parameters.QAM = 64; %Rectangular QAM modualtor, M-ary number  
parameters.SNR = SignaltoNoise; %Signal to noise ratio  
parameters.InputPower = Inputpwr;  
parameters.ViterbiDepth = 34; %Tradeback depth in the viterbi decoder  
parameters.SizeSpectrumScope = parameters.NTotal * ...  
    parameters.OFDMTotalSymPerFrame; %size of the spectrum scope's  
buffer
```


9.2 APPENDIX 2

```

/*****
FILE
Functionsfft.c – Bit reversed function(bit_rev) and fft.
*****/

```

```

#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

```

```

#ifndef PI
# ifdef M_PI
#  define PI M_PI
# else
#  define PI 3.14159265358979323846
# endif
#endif

```

```

bit_rev(float* x, int n)
{
    int i, j, k;
    float rtemp, itemp;
    j = 0;
    for(i=1; i < (n-1); i++)
    {
        k = n >> 1;
        while(k <= j)
        {
            j -= k;
            k >>= 1;
        }
        j += k;
        if(i < j)
        {
            rtemp = x[j*2];

```

```

        x[j*2] = x[i*2];
        x[i*2] = rtemp;
        itemp = x[j*2+1];
        x[j*2+1] = x[i*2+1];
        x[i*2+1] = itemp;
    }
}

void fft(float* x, float* w, short n)
{
    short n2, ie, ia, i, j, k, m;
    float rtemp, itemp, c, s;
    n2 = n;
    ie = 1;

    for(k=n; k > 1; k >>= 1)
    {
        n2 >>= 1;
        ia = 0;

        for(j=0; j < ie; j++)
        {
            c = w[2*j];
            s = w[2*j+1];
            for(i=0; i < n2; i++)
            {
                m = ia + n2;
                rtemp = c * x[2*m] + s * x[2*m+1];
                itemp = c * x[2*m+1] - s * x[2*m];
                x[2*m] = x[2*ia] - rtemp;
                x[2*m+1] = x[2*ia+1] - itemp;
                x[2*ia] = x[2*ia] + rtemp;
                x[2*ia+1] = x[2*ia+1] + itemp;
                ia++;
            }
            ia += n2;
        }
        ie <<= 1;
    }
}

```

```
/******  
FILE  
fft_main.c  
*****/  
  
#include "parameters.h"  
  
void main(void)  
{  
  
    bit_rev(W, N>>1);  
    fft(X, W, N);  
    bit_rev(X, N);  
  
}
```

```

/*****
FILE
parameters.h - This is the C header file for FFT implementation.
N-> Number of FFT points
W->twiddle-factor array
X->Input signal
*****/

const N = 64; /* number of FFT points */

float X[]={1,0,
0.95106, 0,
0.80902,0,
0.58779,0,
0.30902,0,
6.1232e-017,0 ,
-0.30902,0,
-0.58779,0,
-0.80902,0,
-0.95106,0,
-1,0,
-0.95106,0,
-0.80902,0,
-0.58779,0,
-0.30902,0,
-1.837e-016,0,
0.30902,0,
0.58779,0,
0.80902,0,
0.95106,0,
1,0,
0.95106,0,
0.80902,0,
0.58779,0,
0.30902,0,
3.0616e-016,0,
-0.30902,0,
-0.58779,0,
-0.80902,0,
-0.95106,0,
-1,0,
-0.95106,0,
-0.80902,0,
-0.58779,0,
-0.30902,0,

```

```
-4.2863e-016,0,  
0.30902,0,  
0.58779,0,  
0.80902,0,  
0.95106,0,  
1,0,  
0.95106,0,  
0.80902,0,  
0.58779,0,  
0.30902,0,  
5.5109e-016,0,  
-0.30902,0,  
-0.58779,0,  
-0.80902,0,  
-0.95106,0,  
-1,0,  
-0.95106,0,  
-0.80902,0,  
-0.58779,0,  
-0.30902,0,  
-2.4499e-015,0,  
0.30902,0,  
0.58779,0,  
0.80902,0,  
0.95106,0,  
1,0,  
0.95106,0,  
0.80902,0,  
0.58779,0};
```

```
float W[] = {1, 0.19509 ,  
0.92388,0.55557,  
0.70711,0.83147,  
0.38268,0.98079,  
6.1232e-017, 0.98079,  
-0.38268, 0.83147,  
-0.70711, 0.55557,  
-0.92388, 0.19509,  
-1, -0.19509,  
-0.92388,-0.55557,  
-0.70711,-0.83147,  
-0.38268, -0.98079,  
-1.837e-016, -0.98079,  
0.38268, -0.83147,  
0.70711, -0.55557,
```

```
0.92388, -0.19509,  
1, 0.19509,  
0.92388, 0.55557,  
0.70711, 0.83147,  
0.38268, 0.98079,  
3.0616e-016, 0.98079,  
-0.38268, 0.83147,  
-0.70711, 0.55557,  
-0.92388, 0.19509,  
-1, -0.19509,  
-0.92388, -0.55557,  
-0.70711, -0.83147,  
-0.38268, -0.98079,  
-4.2863e-016, -0.98079,  
0.38268, -0.83147,  
0.70711, -0.55557,  
0.92388, -0.19509};
```

På svenska

Detta dokument hålls tillgängligt på Internet – eller dess framtida ersättare – under en längre tid från publiceringsdatum under förutsättning att inga extra-ordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns det lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>

In English

The publishers will keep this document online on the Internet - or its possible replacement - for a considerable time from the date of publication barring exceptional circumstances.

The online availability of the document implies a permanent permission for anyone to read, to download, to print out single copies for your own use and to use it unchanged for any non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional on the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its WWW home page: <http://www.ep.liu.se/>

© Luis Zarzo Fuertes